

TOHOKU UNIVERSITY  
GRADUATE SCHOOL OF INFORMATION SCIENCES

**Memory-aware Task Mapping for Heterogeneous  
Multi-Core Systems**

(不均質マルチコアシステムのためのメモリを考慮したタスクマッピング  
に関する研究)

A thesis submitted for the degree of Master of Science (Information Sciences)

Department of Computer and Mathematical Sciences

by

Yifan JIN

January 27, 2022



# Memory-aware Task Mapping for Heterogeneous Multi-Core Systems

Yifan JIN

## Abstract

As a new trend in processor design, the multi-core processor has evolved to employ a heterogeneous multi-core architecture integrating several kinds of cores with different performance and energy characteristics on a single chip. In addition, the Non-Uniform Memory Access (NUMA) architecture has become a de facto standard in modern HPC systems. Almost all large-scale computing systems can be regarded as NUMA systems. The modern HPC systems are increasingly sensitive to energy consumption. Thus, the deployment of multi-core heterogeneous processors with higher energy efficiency in future large-scale HPC systems is foreseeable. In such a system, in order to avoid the von Neumann bottleneck, its memory subsystem will also be based on the NUMA architecture. The combined use of the two techniques presents new challenges for task mapping on this new system configuration.

Because task mapping could significantly affect the usage of systems' heterogeneity and memory resources, proper task mapping will be a key to high performance and energy efficiency. For heterogeneous multi-core systems, inappropriate task mapping will cause serious load imbalance and waste of system resources. Especially for applications involving different load tasks, choosing the wrong core type for the task will significantly increase execution time and wasted clock cycles, resulting in lower energy efficiency. For the NUMA architecture, if the tasks of implicit communication by reading and writing memory are assigned to different NUMA nodes, the core group sharing memory resources, it will need to transmit data through the interconnection between NUMA nodes, thus suffering from remote access penalty. On the other hand, concentrating the memory access behavior of tasks on the same NUMA node will cause congestion of memory devices, which will also reduce the performance of the system. Introducing heterogeneity into a NUMA system would make task mapping more complex. The mapping strategy not only needs to adjust task mapping among NUMA nodes to optimize the use of memory resources but also needs to select the appropriate core type for each task to optimize the use of processing units.

This work focuses on NUMA systems built with heterogeneous multi-core processors. Most of the conventional studies on NUMA awareness mainly consider homogeneous processors and/or processor cores, while most of the conventional studies on heterogeneous multi-core architectures mainly assume a homogeneous interconnection among cores. However, due to the emergence of the new system configuration, a new task mapping technique that simultaneously considers both two factors, NUMA memory awareness and core heterogeneity, is necessary. The objective of this work is to deal with the task mapping problem on the new system configuration by properly combining memory-aware task mapping and heterogeneity-aware task mapping.

Considering the diversity of system configuration and application characteristics, this work proposes a task mapping strategy that switches between two priority options to determine the best mapping for the target application on the target system. The workflow of the proposed task mapping includes analyzing application characteristics and system configuration, then selecting an appropriate priority option for the current situation, and finally using the selected priority option to complete task mapping. The two proposed priority options are the memory-aware priority option (MPO) and the heterogeneity-aware priority option (HPO). Choosing one of the priority options will prioritize the impact of that factor at the task mapping. However, it does not mean that the influence of the other factor on the mapping result is completely ignored because some applications might be sensitive to both memory and core performance. The proposed priority option switching mechanism (POSM) selects appropriate priority options for individual systems and applications considering their performance characteristics.

In the preliminary evaluation, this work uses simulations to compare the performance and energy efficiency changes of applications with different characteristics on the target system. The experimental results show that when using HPO, only some applications show an improvement over the baseline, called

the round-robin mapping. As also shown in the previous work, MPO is not suitable for all applications. These two results have shown that one priority option alone cannot provide a suitable mapping for all kinds of applications. This shows the necessity of the two mapping priority options and the switching mechanism proposed in this work.

To select the appropriate priority option, it is necessary to investigate the key performance characteristics of the applications. To do this, this work proposes two potential benefits values: memory-aware potential benefit and heterogeneity-aware potential benefit. These two values are used to quantify the impact of one-kind mapping optimization on performance metrics. The larger the potential benefit value, the greater the benefit obtained by selecting this kind of mapping optimization. After normalizing these two values, they are used to guide POSM proposed in this work. For a combination of an application and a system, the two potential benefits values are calculated, and the switching mechanism will select the priority option corresponding to the larger benefit value.

This work also evaluates the effectiveness of the proposed task-mapping priority options and the accuracy of POSM. The evaluation results show that the proposed POSM can accurately select appropriate mapping priority options after analyzing application characteristics and system configurations. After applying the selected mapping priority option for task mapping, the applications can gain improvements in energy efficiency or memory access performance. Compared with other techniques that do not switch mapping priorities, the technique proposed in this work can achieve overall improvements in energy efficiency and memory access performance when dealing with a set of applications with different characteristics.

# Contents

## Abstract

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                | <b>1</b>  |
| 1.1      | Background . . . . .                                               | 1         |
| 1.2      | Motivation and Objective . . . . .                                 | 5         |
| 1.3      | Thesis Organization . . . . .                                      | 7         |
| <b>2</b> | <b>Task Mapping Techniques</b>                                     | <b>8</b>  |
| 2.1      | Introduction . . . . .                                             | 8         |
| 2.2      | Memory-aware Task Mapping . . . . .                                | 9         |
| 2.3      | Heterogeneity-aware Task Mapping . . . . .                         | 10        |
| 2.4      | Multi-objective Task Mapping . . . . .                             | 12        |
| 2.5      | Conclusion . . . . .                                               | 13        |
| <b>3</b> | <b>Task Mapping with Considering Both Memory and Heterogeneity</b> | <b>14</b> |
| 3.1      | Introduction . . . . .                                             | 14        |
| 3.2      | Overview . . . . .                                                 | 14        |
| 3.3      | Task Mapping Strategy with Two Priority Options . . . . .          | 17        |
| 3.3.1    | Memory-aware Priority Option (MPO) . . . . .                       | 17        |
| 3.3.2    | Heterogeneity-aware Priority Option (HPO) . . . . .                | 21        |
| 3.4      | Priority Option Switching Mechanism . . . . .                      | 24        |
| 3.5      | Conclusion . . . . .                                               | 29        |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>4</b> | <b>Evaluation</b>                                       | <b>30</b> |
| 4.1      | Introduction . . . . .                                  | 30        |
| 4.2      | Experimental Environment . . . . .                      | 30        |
| 4.3      | Preliminary Evaluation . . . . .                        | 34        |
| 4.3.1    | The necessity for heterogeneity . . . . .               | 34        |
| 4.3.2    | The importance of suitable task mapping . . . . .       | 36        |
| 4.3.3    | The necessity of priority switching mechanism . . . . . | 37        |
| 4.4      | Mapping Strategy Evaluation . . . . .                   | 38        |
| 4.4.1    | Memory-aware mapping . . . . .                          | 39        |
| 4.4.2    | Heterogeneity-aware mapping . . . . .                   | 43        |
| 4.5      | Switching Mechanism Evaluation . . . . .                | 44        |
| 4.5.1    | Effectiveness of the Switching mechanism . . . . .      | 45        |
| 4.5.2    | Accuracy of the Switching mechanism . . . . .           | 45        |
| 4.6      | Conclusion . . . . .                                    | 49        |
| <b>5</b> | <b>Conclusions</b>                                      | <b>50</b> |
|          | <b>Bibliography</b>                                     | <b>54</b> |
|          | <b>Publication</b>                                      | <b>58</b> |
|          | <b>Acknowledgements</b>                                 | <b>59</b> |

# List of Figures

|     |                                                                                                                             |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Overview of Apple M1 series processor [4]. . . . .                                                                          | 2  |
| 1.2 | Overview of Intel Alder Lake series processor [14]. . . . .                                                                 | 2  |
| 1.3 | Comparison of UMA and NUMA architectures. . . . .                                                                           | 4  |
| 3.1 | Workflow of the proposed mapping strategy. . . . .                                                                          | 15 |
| 3.2 | A NUMA system built heterogeneous multi-core processors. . . . .                                                            | 16 |
| 3.3 | Memory-aware priority task mapping process. . . . .                                                                         | 18 |
| 3.4 | Memory-aware priority task mapping example. . . . .                                                                         | 20 |
| 3.5 | Heterogeneity-aware priority task mapping process. . . . .                                                                  | 22 |
| 3.6 | Heterogeneity-aware priority task mapping example. . . . .                                                                  | 23 |
| 3.7 | Priority option switching mechanism concept. . . . .                                                                        | 24 |
| 3.8 | Priority option switching mechanism process. . . . .                                                                        | 27 |
| 4.1 | Comparison of CPI stacks when executing the Rodinia-lavaMD bench-<br>mark on homogeneous and heterogeneous systems. . . . . | 35 |
| 4.2 | Performance and energy comparison of homogeneous and heteroge-<br>neous systems. . . . .                                    | 35 |
| 4.3 | Performance and energy impacts of using different mapping strategies. .                                                     | 37 |
| 4.4 | Performance and energy impacts of using the heterogeneity-aware pri-<br>ority on different applications. . . . .            | 38 |
| 4.5 | NPB-BT task communication matrix. . . . .                                                                                   | 39 |
| 4.6 | NPB-BT task memory access behavior. . . . .                                                                                 | 40 |

|      |                                                                                 |    |
|------|---------------------------------------------------------------------------------|----|
| 4.7  | Performance improvements from memory-aware mapping optimization.                | 40 |
| 4.8  | Comparison of the total communication volume of each task. . . . .              | 41 |
| 4.9  | Comparison of the total amount of memory accessed by each task. . . .           | 42 |
| 4.10 | Sniper profile execution output file. . . . .                                   | 42 |
| 4.11 | Performance improvements from heterogeneity-aware mapping optimization. . . . . | 43 |
| 4.12 | Comparison of relative tasks load. . . . .                                      | 44 |
| 4.13 | Normalized execution time for each application. . . . .                         | 46 |
| 4.14 | Sum of normalized execution times. . . . .                                      | 47 |
| 4.15 | Normalized PBs for each application. . . . .                                    | 48 |



# List of Tables

|     |                                                 |    |
|-----|-------------------------------------------------|----|
| 4.1 | Heterogeneous simulation configuration. . . . . | 33 |
| 4.2 | Homogeneous simulation configuration. . . . .   | 33 |
| 4.3 | Benchmark application characteristics. . . . .  | 34 |

# List of Abbreviations

|             |                                                                     |
|-------------|---------------------------------------------------------------------|
| <b>SMP</b>  | <b>S</b> ymmetric <b>M</b> ulti <b>P</b> rocessing                  |
| <b>AMP</b>  | <b>A</b> symmetric <b>M</b> ulti <b>P</b> rocessing                 |
| <b>NUMA</b> | <b>N</b> on- <b>U</b> niform <b>M</b> emory <b>A</b> ccess          |
| <b>ILP</b>  | <b>I</b> nstruction- <b>L</b> evel <b>P</b> arallelism              |
| <b>ISA</b>  | <b>I</b> nstruction <b>S</b> et <b>A</b> rchitecture                |
| <b>MPO</b>  | <b>M</b> emory-aware <b>P</b> riority <b>O</b> ption                |
| <b>HPO</b>  | <b>H</b> eterogeneity-aware <b>P</b> riority <b>O</b> ption         |
| <b>POSM</b> | <b>P</b> riority <b>O</b> ption <b>S</b> witching <b>M</b> echanism |
| <b>PB</b>   | <b>P</b> otential <b>B</b> enefits                                  |
| <b>LLC</b>  | <b>L</b> ast <b>L</b> evel <b>C</b> ache                            |
| <b>MC</b>   | <b>M</b> emory <b>C</b> ontroller                                   |
| <b>CPI</b>  | <b>C</b> lock <b>P</b> er <b>I</b> nstruction                       |
| <b>IPC</b>  | <b>I</b> nstruction <b>P</b> er <b>C</b> lock                       |

# Chapter 1

## Introduction

### 1.1 Background

With the end of Moore's Law [21], researchers who try to improve the performance of computing systems have to turn their attention from increasing the clock frequency of the CPU core to the deployment of multi-core architectures and optimization of the memory subsystem.

Developments in these two directions represent new trends. The multi-core processor has evolved from a traditional multi-core structure containing homogeneous cores to a heterogeneous multi-core architecture integrating cores with different performance characteristics on a single chip [17]. The optimization of the memory subsystem is no longer limited to widening the bandwidth of the memory bus in the Symmetric Multiprocessing (SMP) system. By introducing the Non-uniform Memory Access (NUMA) architecture, the system will provide separate memory devices for each processor to alleviate the von Neumann bottleneck caused by the shared bus limitation [18].

There are two main architectures of modern multi-core processors: homogeneous multi-core and heterogeneous multi-core. The homogeneous multi-core architecture includes only identical cores. Although it is the earliest and most widely utilized architecture in multi-core processor design, the homogeneous multi-core architecture has

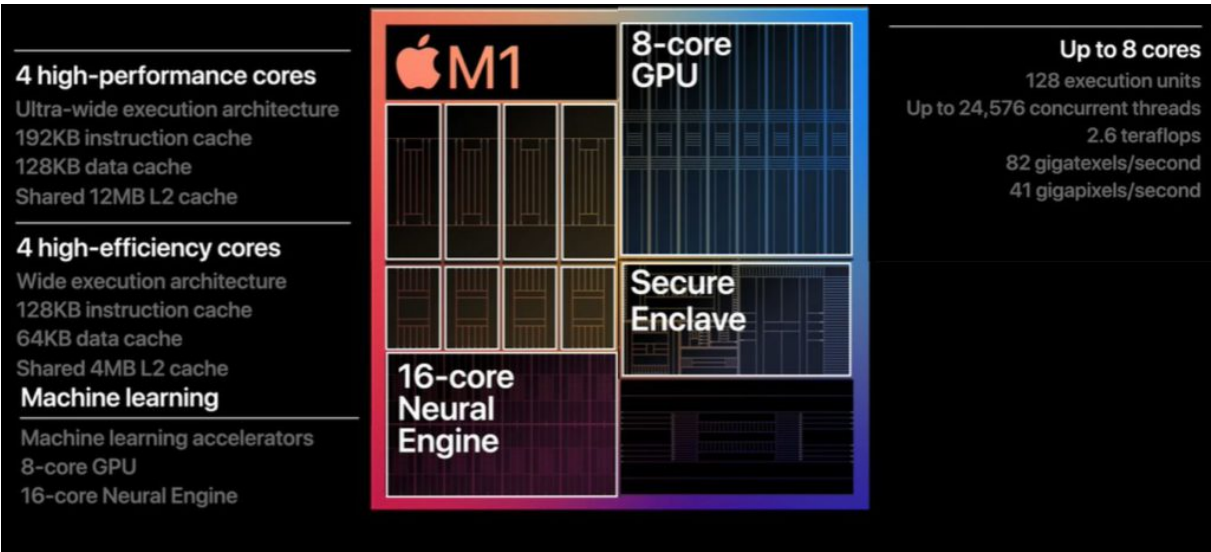


FIGURE 1.1: Overview of Apple M1 series processor [4].

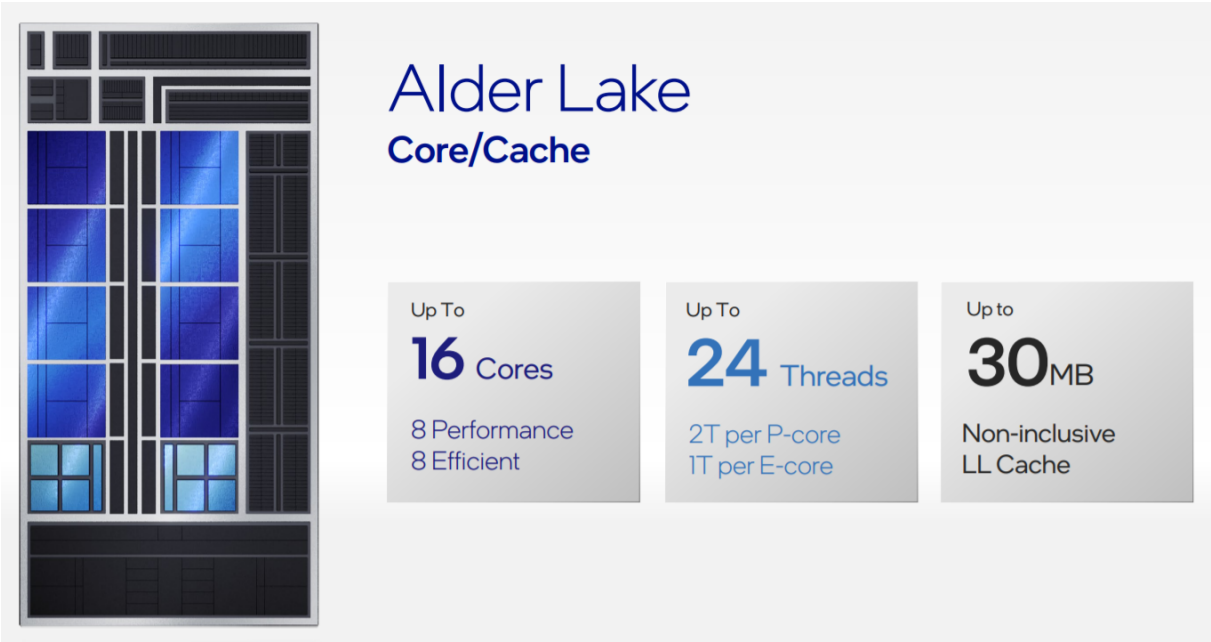


FIGURE 1.2: Overview of Intel Alder Lake series processor [14].

some disadvantages. Kumar et al. [17] pointed out that different applications may have different resource requirements during their execution. Some applications may have a large amount of instruction-level parallelism (ILP), which can be exploited by a core that can issue many instructions per cycle. The same core, however, might waste its resources when running an application with little ILP, and consume significantly more power than a simpler core that is better matched to the characteristics of the application.

To overcome the disadvantages of the homogeneous multi-core architecture, the heterogeneous multi-core architecture was proposed. The heterogeneous multi-core architecture contains cores that are not identical. By appropriately adapting different cores for different kinds of applications, this architecture can achieve performance and energy efficiency improvements. Depending on whether different core types use the same Instruction Set Architecture (ISA), the heterogeneous multi-core architecture can be divided into the Multi-ISA heterogeneous multi-core architecture and the Single-ISA heterogeneous multi-core architecture. In contrast to Multi-ISA architecture in which different cores use different ISAs, all cores use the same ISA in the Single-ISA architecture. Single-ISA heterogeneous multi-core architecture reduces the complexity of programming and compilation compared with heterogeneous systems that require offloading programming models because of the use of different ISAs. The omission of the offloading process also reduces the transmission cost of transferring large amounts of data between the CPU and the accelerator.

Recently, several commercial processor manufacturers have introduced new heterogeneous multi-core processors. For example, the Apple M1 series processors shown in Figure 1.1 contain four "high-performance cores" and four "high-efficiency cores". Another example is the Intel Alder Lake series processors shown in Figure 1.2, which contain eight "performance cores" and eight "efficiency cores". Although different names

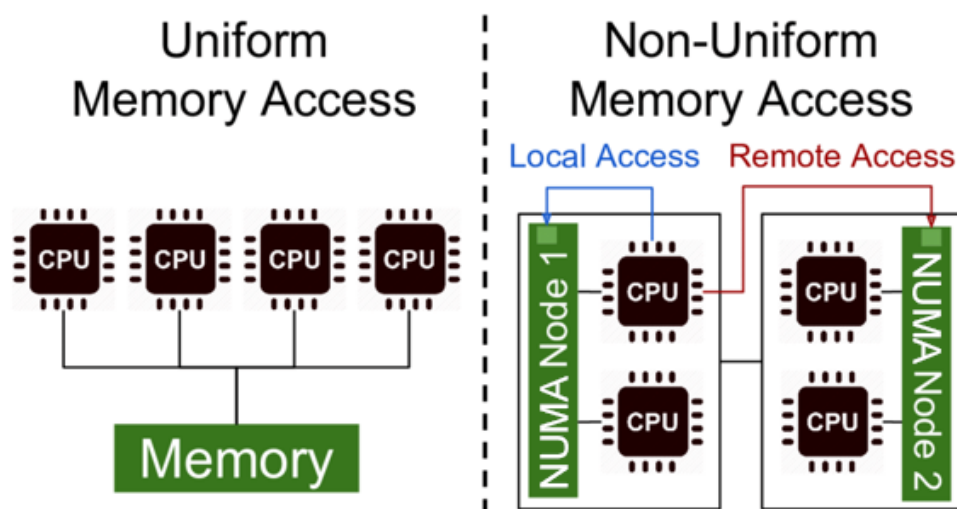


FIGURE 1.3: Comparison of UMA and NUMA architectures.

are used to describe the two core types, it can be seen that the mainstream heterogeneous multi-core processors have the following characteristics: cores use a single ISA, are divided into high-performance cores and low-performance cores, and multiple cores share the same Last Level Cache (LLC). When manufacturers introduced these processors, in addition to performance improvements, they emphasized the huge improvements these processors can achieve in terms of energy efficiency [4][14]. This is because in systems using these processors, non-urgent applications will be executed on the more energy-efficient cores. Given that modern HPC systems are tightly constrained by energy consumption, the heterogeneous multi-core architecture that improves energy efficiency shows significant practical value in HPC.

In addition, the NUMA architecture has become a de facto standard in modern HPC systems [23]. In contrast with the Uniform Memory Access (UMA), the memory access of the processor in the NUMA system is "non-uniform", which indicates that the memory access latency and bandwidth depend on the memory location relative to the processor. In a modern NUMA system, a group of processor cores shares memory devices such as LLCs and memory controllers. Such a group of processors is referred

to as a NUMA node. As shown in Figure 1.3, the processor core not only accesses the memory devices of its own node but also accesses the memory devices of other nodes connected by the interconnect. The memory access behavior of the processor core includes local access and remote access. Local access refers to accessing memory devices that reside within the same NUMA node as the core; while remote access refers to accessing memory devices on other NUMA nodes through interconnection. Because remote access needs to transmit data over the interconnection, this type of access has a higher latency, which is also known as the remote access penalty.

To avoid remote access penalty, researchers have been trying to reduce the number of remote accesses in the system. Because remote accesses are mainly caused by the explicit or implicit communication between two tasks distributed on different NUMA nodes, properly adjusting the communication behavior reduces remote access. Thus, some studies have proposed assigning tasks with dense communication behavior to the same NUMA node aiming to maximize the locality of memory access [15][20][25]. However, other studies later pointed out that maximizing locality may cause memory congestion, which refers to congestion on shared LLCs and memory controllers due to a large number of memory accesses from cores [2][10][13]. Therefore, proper management of memory resources is crucial to obtain good performance on modern NUMA systems.

## 1.2 Motivation and Objective

Based on the background above, it is reasonable to believe that a new HPC system architecture combining the heterogeneous multi-core architecture and NUMA architecture will emerge in the future. In such systems, multiple heterogeneous cores are integrated in a NUMA node and share memory devices such as LLC and memory controller. Depending on the location of the memory, memory access is divided into local

access and remote access.

However, existing task mapping techniques are ineffective on these future systems that employ both heterogeneous multi-core architecture and NUMA architecture. Most conventional studies on locality and memory congestion assume homogeneous multi-core systems. The introduction of heterogeneous processors into the NUMA system will require task mapping to further consider the performance differences of different cores. On the other hand, most conventional studies on heterogeneous multi-core processors assume a homogeneous interconnection among cores. Existing techniques that deal with task-to-core allocation on heterogeneous multi-core processors will no longer provide optimal solutions when communication efficiency and thread migration costs become inconsistent across cores.

Therefore, the objective of this thesis is to tackle the task mapping problem on the new system configuration by properly combining memory-aware task mapping and heterogeneity-aware task mapping. This work considers data locality, memory congestion, and processor performance differences at the same time in order to provide a suitable task mapping on such heterogeneous NUMA systems.

The main contributions of this thesis are summarized as follows:

1. This thesis analyzes the application prospects of heterogeneous multi-core processors in the field of HPC.
2. This thesis identifies the new challenges that introducing heterogeneous multi-core processors into existing NUMA systems will bring to task mapping.
3. This thesis proposes a task mapping strategy that switches between two priority options to determine the best mapping for the target application on NUMA systems built with heterogeneous multi-core processors.
4. This thesis investigates the key performance characteristics of applications that can be used to determine the mapping priority.



## 1.3 Thesis Organization

The rest of this thesis is organized as follows:

Chapter 2 introduces two types of mapping techniques: memory-aware task mapping and heterogeneity-aware task mapping. Through the analysis of the work related to these two techniques, the objectives of the mapping strategy part of this work are identified. At the same time, this chapter briefly describes another work similar to this work and emphasizes the difference between the two works and the innovation of this work.

Chapter 3 describes the mapping technique proposed in this work in detail, including a mapping strategy with two priority options and a priority option switching mechanism. The workflows of the mapping strategy, switching mechanism, and the corresponding mapping example are given.

Chapter 4 presents the evaluation results. It includes the preliminary evaluation to demonstrate the necessity of the priority switching mechanism proposed in this study, the evaluation of the effectiveness of the two priority mapping strategies, and the evaluation of the effectiveness and the accuracy of the priority option switching mechanism.

Chapter 5 summarizes the thesis and briefly describes the future work.

## Chapter 2

# Task Mapping Techniques

### 2.1 Introduction

Task mapping is an important part of resource management, and refers to the process of mapping tasks generated by applications to processing units in the system. The task mapping process, which considers the performance and characteristics of various components in the computing system, has been extensively studied. To deal with task mapping, researchers have considered a variety of factors and performance metrics.

One type of widely adopted task mapping techniques considers the performance gap between processors and memory devices. In this thesis, this type of mapping is referred to as memory-aware task mapping. Another type of task mapping techniques optimizes for heterogeneous multi-core systems. On heterogeneous multi-core systems, it is also necessary to map the tasks to the appropriate cores in order to improve performance. In this thesis, this type of mapping is referred to as heterogeneity-aware task mapping.

Sections 2.2 and 2.3 review the existing work on the above two types of mappings, and summarize their core ideas. Furthermore, multi-objective task mapping techniques that consider multiple factors in task mapping are also reviewed.

## 2.2 Memory-aware Task Mapping

The emergence of NUMA architecture, deep cache hierarchy, and other technologies increasing the complexity require researchers to carefully consider the memory access behavior of applications and the memory structure of the target system when designing task mapping techniques.

In a NUMA system, mapping tasks of an application to NUMA nodes, the core group sharing memory resources, has a significant impact on performance including execution time and energy efficiency. Because the mapping affects the communication behaviour among tasks and will result in different communication costs. The communication between tasks is achieved through memory access. Therefore, the memory-aware task mapping usually pays attention to tune the communication behavior of the tasks. The memory-aware task mapping tries to avoid situations that will negatively affects system performance, such as remote access or memory congestion.

In order to avoid remote access, a mapping strategy to improve the locality of memory access is proposed. This strategy ensures that most of the communication occurs within NUMA nodes by aggregating tightly communicating groups of tasks on the same NUMA node. As an example, Jeannot et al. [15] proposed TreeMatch, a new mapping algorithm, to compute a task mapping tailored for the target system. It considers both the application communication pattern and the system hardware topology. By matching the communication pattern to the underlying system hardware topology, they decreased parallel applications communication costs and ultimately improved the performance of systems.

Meanwhile, several studies [10][13][25] showed that maximizing the locality can cause memory congestion and degrade performance. Memory congestion is mainly caused by a large number of concurrent communications occurring on a single node. In other words, memory congestion happens when the requirements of the core on

the shared memory device on the NUMA node exceed their load capacity. To prevent memory congestion, the load of shared memory devices on a single NUMA node can be reduced by distributing tasks that tend to communicate at the same time to different NUMA nodes. Diener et al. [11] presented a mapping technique that increases locality while still maintaining the balance in memory access. The evaluation on a real system showed that their mapping technique can improve performance and avoids the performance losses that may occur using a pure locality-based mapping technique.

Similarly, Agung et al. [1] proposed a task mapping technique, DeLoc, to address both locality and memory congestion. Their mapping technique analyzes the spatial and temporal communication behaviors of parallel applications to identify groups of tasks that potentially cause memory congestion. They also introduced metrics to describe the communication behaviors and determine if a parallel application can benefit from locality and memory-congestion-aware task mapping.

Based on the above-mentioned memory-aware mapping technology, this work discusses the way of avoiding memory congestion while maintaining certain memory access locality.

## **2.3 Heterogeneity-aware Task Mapping**

A heterogeneous platform usually refers to a set of processing units of different types within a computing system. Andrade et al. [3] pointed out that multiple studies refer to this term in different ways. In addition to referring to a system with different types of processors, this term also refers to platforms containing processors of the same type, but with different performance. As stated in the title of this thesis, this study only considers heterogeneous multi-core processors. Specifically, the type of processor and other characteristics are not considered, and only the differences in processor performance are discussed.

Heterogeneity-aware task mapping techniques usually focus on selecting the appropriate core type for the task to improve performance and energy efficiency. In the literature, Kumar et al. [17] were the first to demonstrate the potential improvement in energy efficiency with minor performance loss via dynamic workload mapping for heterogeneous multi-core systems. Koufaty et al. [16] proposed bias scheduling that inspects the Clock Per Instruction (CPI) of a program and classifies a task as either "little-core" biased or "big-core" biased based on the ratio of cycles spent on execution and stalls. By monitoring application bias, the operating system is able to match tasks to the core type that can maximize system throughput. Poovey et al. [22] considered thread behavior that results from parallel programs and proposed four thread mapping techniques for the heterogeneous multi-core architecture. These techniques leverage the behavior of common programming patterns used in parallel programming to target typical thread behavior. The authors also demonstrated the relationship between thread heterogeneity and the parallel pattern used to create the workload. Chen et al. [9] presented a technique to map application to its optimum core by analyzing the micro-architecture independent characteristics of that application. This technique exploits data or instruction reuse distance and true dependency distance to derive switching gains and ranks for each configuration, and maps the application to the optimum core accordingly.

Based on the above-mentioned heterogeneity-aware mapping technology, in this work, the mapping of tasks to heterogeneous cores will follow the principle of mapping high-load tasks to high-performance cores. This is also because some preliminary experiments show that such a mapping strategy can yield steady performance improvements in execution time.

## **2.4 Multi-objective Task Mapping**

The two types of mapping strategies reviewed in the previous two sections usually do not simultaneously take two criteria into account. That is, research on memory-aware task mapping usually assumes that the processor cores in the system are homogeneous. On the other hand, research on heterogeneity-aware task mapping usually does not discuss the impact of changing the mapping on memory access. This thesis considers the two factors of memory structure and processor core heterogeneity when finding the task mapping. Similar to this work, there are few studies that consider multiple factors at the same time.

Ding et al. [12] proposed a heterogeneity-aware schedule technique. They introduced a new metric called Online Asymmetric Multi Processing (AMP) Speedup Factor to define which processes should run on fast cores. The Online AMP Speedup Factor took the characteristics of a process and the thread-level parallelism into account. The experimental results showed the metric can help systems choose the correct core type. Moreover, because the processes would degrade its performance if they do not scatter and thereby could not get the wanted resource in time. The authors also proposed a new metric called Contention Degradation Factor to quantify which processes consume the most resources.

The most significant difference between the prior multi-objective optimization mapping techniques and this work is that the above research will optimize the mapping in a fixed order. That is, in the prior methods of using two factors, one factor is considered first to create an initial mapping, and then the other factor is considered to improve the mapping. The priority option switching mechanism proposed in this study will select the appropriate priority option for each independent application. Different priority options will reverse the order of memory-aware mapping optimization and heterogeneity-aware mapping optimization. This work expects to adapt to changes

in system configuration parameters and application characteristics by using different priority options to give a more suitable task mapping.

## 2.5 Conclusion

This chapter introduced two types of mapping techniques: memory-aware task mapping and heterogeneity-aware task mapping. The analysis of previous studies related to these two techniques identified the objectives of the mapping strategy part of this work: avoiding memory congestion while ensuring communication locality, and choosing appropriate heterogeneous cores for different tasks. This chapter also briefly introduced the related work of this thesis, and emphasized the differences from the existing studies to clarify the novelty of this thesis. That is, the mapping technique presented in this thesis provides more appropriate mappings for different applications and systems through the priority option switching mechanism.

## Chapter 3

# Task Mapping with Considering Both Memory and Heterogeneity

### 3.1 Introduction

Considering the diversity of system configurations and application characteristics, this thesis proposes a task mapping strategy that switches between two priority options to determine the best mapping for the various applications on the various systems. This chapter details its design principles and workflow.

### 3.2 Overview

The workflow of the task mapping strategy proposed in this work is shown in Figure 3.1. The proposed task mapping strategy uses a switching mechanism to select the appropriate priority option for the target application and target system.

As mentioned in Chapter 2, this thesis focuses on a task mapping strategy that considers the NUMA and core heterogeneity factors. Since a task can only be assigned to one processor core at one time, it is impossible for the mapping strategy to deal with these two factors simultaneously. Moreover, the impacts of the NUMA and core heterogeneity factors may change depending on the application and system. Therefore,



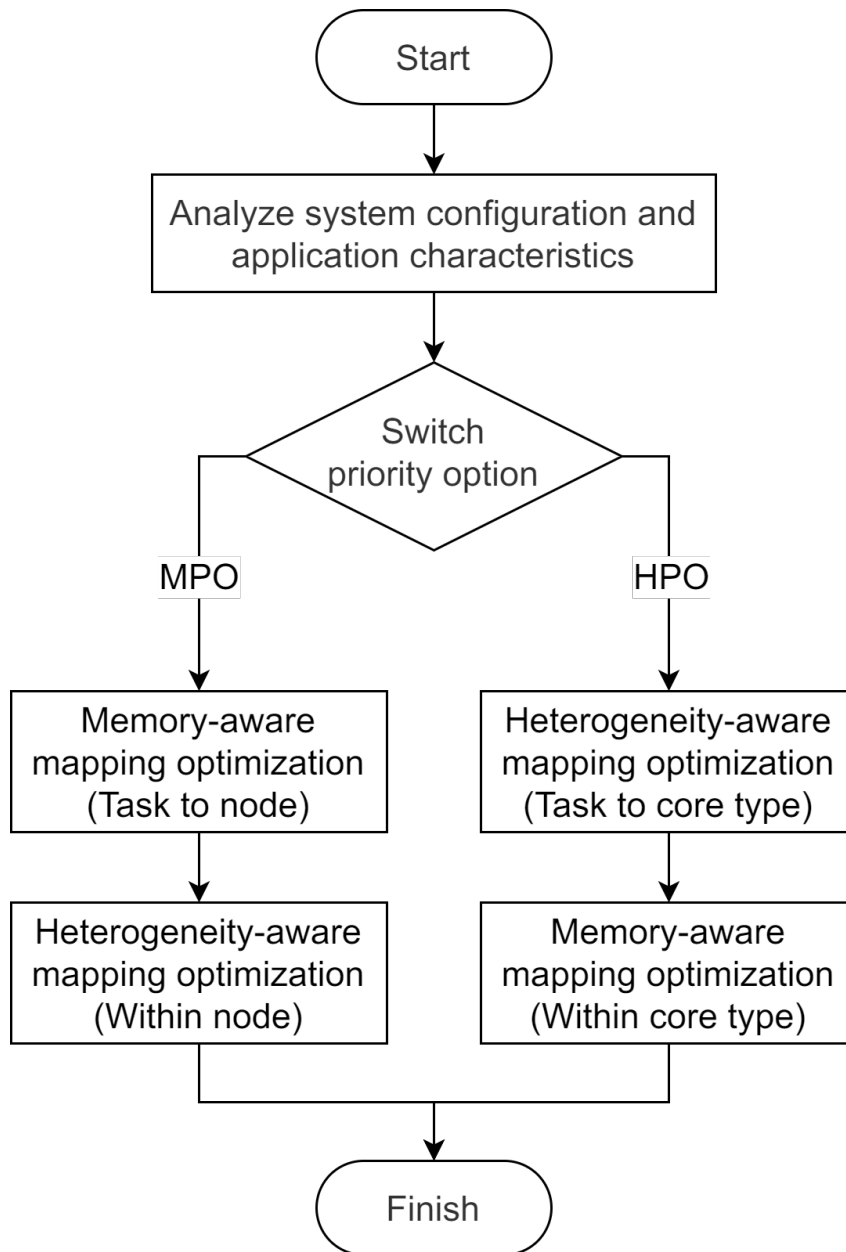


FIGURE 3.1: Workflow of the proposed mapping strategy.

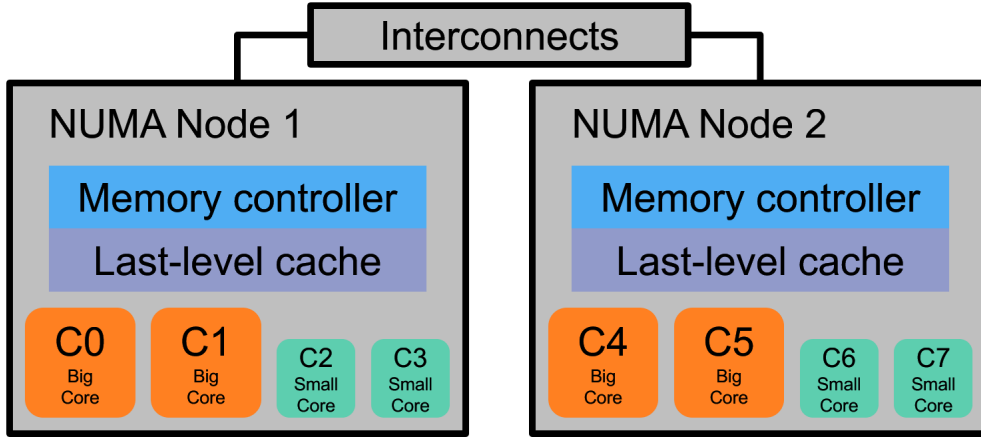


FIGURE 3.2: A NUMA system built heterogeneous multi-core processors.

it is necessary to change the priority order of the two factors depending on the application. As a result, the task mapping results can improve performance and energy efficiency of different applications on the heterogeneous multi-core systems.

As previously mentioned in Chapter 1, this thesis targets task mapping on heterogeneous multi-core systems. Figure 3.2 shows an example of the target systems, which consists of several NUMA nodes, each of which contains two different types of cores. The high-performance cores are hereinafter called "Big cores" and the low-performance cores are called "Small cores". The heterogeneity within a NUMA node comes from a single-ISA heterogeneity multi-core architecture. The reasons for assuming such a target system are: firstly, using identical NUMA nodes allows us to use some memory-aware mapping techniques in existing research; secondly, using the single-ISA can increase the flexibility of task mapping.

As shown in Figure 3.1, the proposed strategy will first analyze the system configuration and application characteristics. In order to provide the necessary information for the mapping process, this work assumes that the system configuration (including the ratio between the number of Big and Small cores, and the performance ratio between a single Big and Small core) and application characteristic information (including the

load information of the tasks generated by the application and the inter-task communication behavior information) are known. This information can be obtained through system topology analysis software, application profilers, communication tracking tools and other methods. Then, using this information, the proposed priority option switching mechanism (POSM) selects the appropriate one from the two priority options. The two proposed priority options are the memory-aware priority option (MPO) and the heterogeneity-aware priority option (HPO). MPO focuses on optimizing the utilization of the NUMA architecture, whereas HPO focuses on optimizing the utilization of core heterogeneity. Choosing one of the priority options will prioritize the impact of that factor when mapping. The influence of another factor on the mapping will not be ignored but secondary. After optimization using MPO or HPO, the strategy will emit a mapping of tasks to cores. The next two sections will detail the proposed MPO, HPO, and POSM.

## 3.3 Task Mapping Strategy with Two Priority Options

The proposed task mapping strategy determines the priority options between MPO and HPO. Choosing one of the priority options will prioritize the impact of that factor during task mapping. However, both mapping options will take into account the two factors when calculating the task mapping. As a result, the proposed task mapping strategy can consider both factors in some applications that are sensitive to both memory and core performance.

### 3.3.1 Memory-aware Priority Option (MPO)

The Memory-aware Priority Option (MPO) is suitable for target systems where the heterogeneity is not obvious, or when there is no difference in the load of tasks generated by the application.

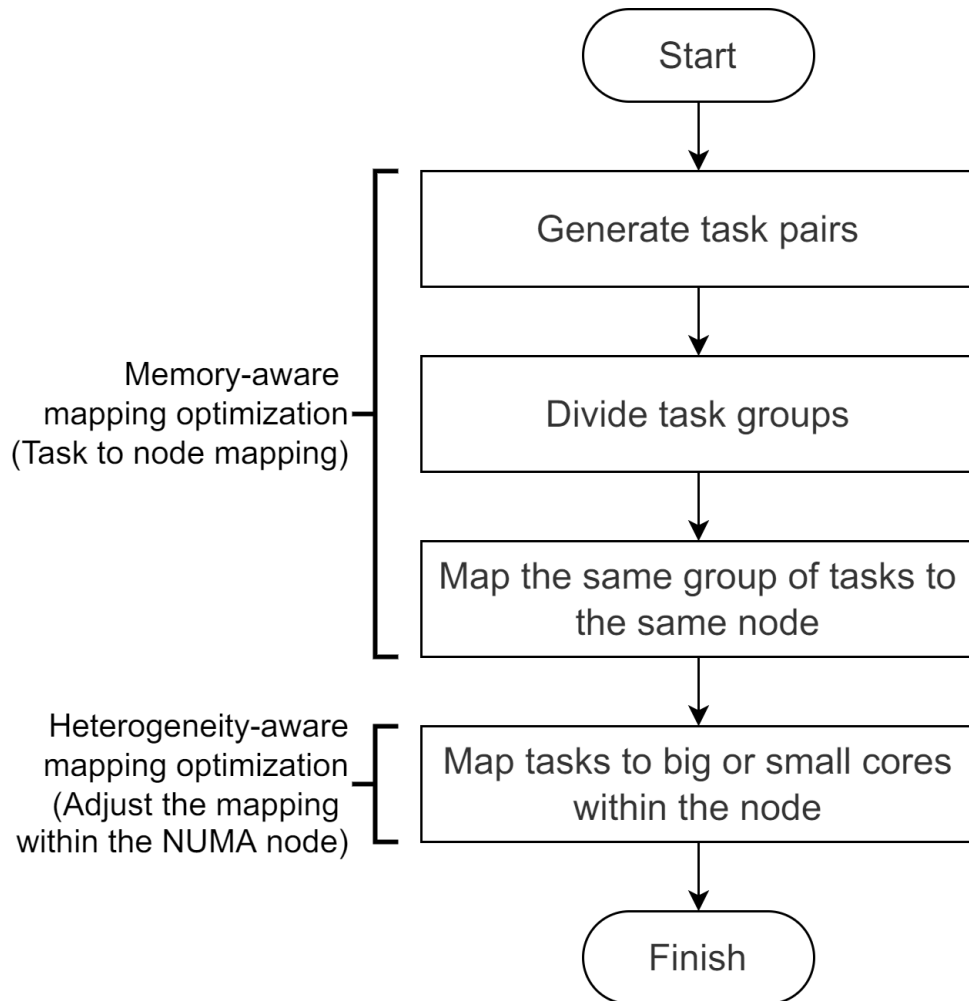


FIGURE 3.3: Memory-aware priority task mapping process.

### 3.3. Task Mapping Strategy with Two Priority Options

---

For example, suppose that an application generates tasks equal to the number of cores in the system, and the load of each task is exactly the same. Then, even if the cores of the target system are heterogeneous, changing the mapping based on heterogeneous multi-core factors does not affect the execution performance. Exchanging tasks on any two cores does not change the overall execution time of the application because the load is exactly the same. The specific task mapping process under MPO is shown in Figure 3.3. Once this priority option is selected, the system will first perform memory-aware mapping optimization.

First, the mapping strategy will pair individual tasks according to the communication behavior information of the tasks to generate task pairs with communication behaviors. A pair of tasks will be mapped to the same NUMA node to avoid remote access and improve locality.

Then, the mapping strategy divides the task pairs obtained in the first step into task groups equal to the number of NUMA nodes in the system according to the communication behavior information of the tasks, and the task pairs in the same task group will be mapped to the same NUMA node. By assigning task pairs within a group to the same NUMA node, this mapping can avoid memory congestion on a particular NUMA node [1]. Memory congestion caused by a large number of concurrent communications can be avoided. At this point, the memory-aware mapping is completed, and the subsequent heterogeneity-aware mapping optimization will be carried out in each NUMA node so that it will not affect the former mapping between tasks to NUMA node.

Finally, the mapping strategy will take into account the core heterogeneity factor by mapping high-load tasks to Big cores and low-load tasks to Small cores within a NUMA node. At this point, the mapping between tasks to cores within each NUMA node is completed, and the entire mapping process finishes.

Figure 3.4 shows an example of using MPO to map eight tasks to eight cores. In the

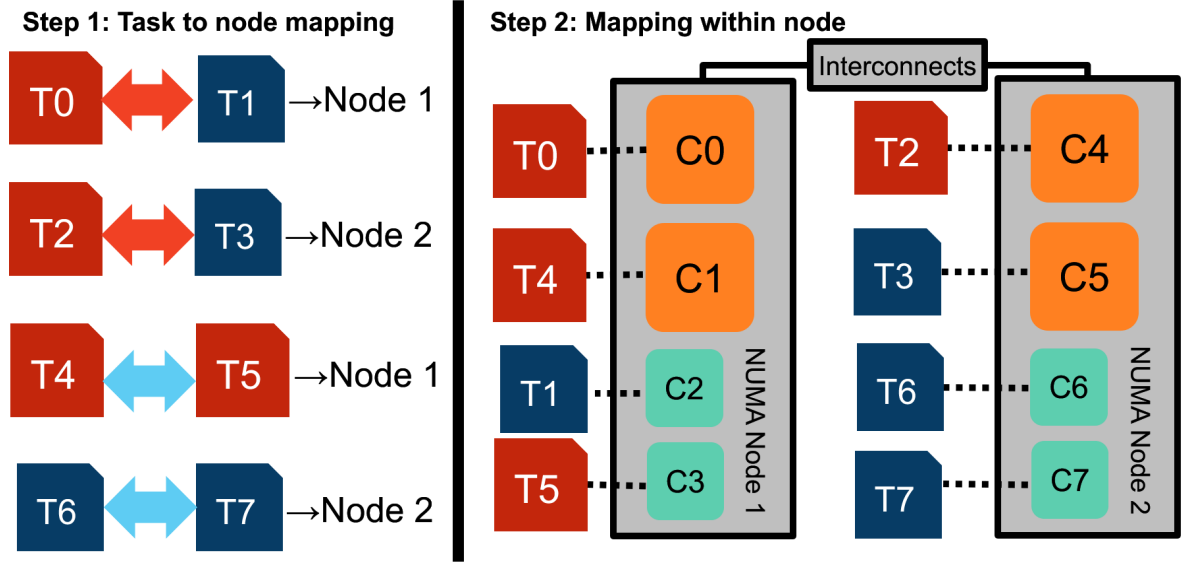


FIGURE 3.4: Memory-aware priority task mapping example.

examples in this subsection and the next subsection, in order to represent task characteristics, the size of task icons is used to indicate the load of the task; the width of the double arrow connecting two tasks is used to indicate the strength of inter-task communication; arrows with the same colors are used to indicate concurrent communication between tasks.

From the task characteristics shown in Figure 3.4, it can be seen that the load difference between tasks included in the application is small, and the communication intensity between tasks is large. Therefore, the proposed mapping strategy uses MPO for task mapping.

In Step 1 of the example, MPO first forms four task pairs according to the communication between tasks, and then divides the four task pairs into two groups, so that task pairs that tend to have concurrent communication are assigned to different groups. This enables intense communications within the same group to occur at different times. Tasks within the same group are assigned to the same node, for example T0, T1, T4 and T5 in the figure are assigned to Node 1. By prioritizing the mapping between tasks and NUMA nodes, MPO can reduce remote access penalties due to

inter-node communication and avoid memory congestion caused by a large number of concurrent communication on the same node.

In Step 2, MPO distributes tasks to processor cores within each NUMA node by assigning high-load and low-load tasks to Big and Small cores, respectively. For example, T0, T1, T4 and T5 are assigned to Node 1. Then, this step tries to assign the high-load tasks, T0, T4 and T5, to the Big cores. However, since there are only two Big cores on each node, MPO has to assign one of the high-load tasks to a Small core. This means that the ideal mapping optimization cannot be obtained at this step. However, since the application contains tasks with relatively small differences in load, using the non-optimal mapping at this step will not seriously affect performance.

#### 3.3.2 Heterogeneity-aware Priority Option (HPO)

The Heterogeneity-aware Priority Option (HPO) is suitable for target systems with obvious heterogeneity, or when the memory access behavior of tasks generated by the application is very small. Although HPO may reduce the memory accesses locality, the performance improvement obtained from heterogeneity-aware mapping can surpass the impacts of memory access locality. In addition, if the application has a small number of memory accesses, aggregating more tasks on the same NUMA node will not cause memory congestion.

Figure 3.5 shows the task mapping steps when the HPO is selected. Once this priority option is selected, the system will first perform heterogeneity-aware mapping optimization.

First, the mapping strategy classifies tasks into two categories, high-load and low-load tasks, which correspond to the Big and Small cores, respectively. Each task is classified by comparing its load to those of the other tasks. At this point, the heterogeneity-aware mapping is completed, and the subsequent memory-aware mapping will be

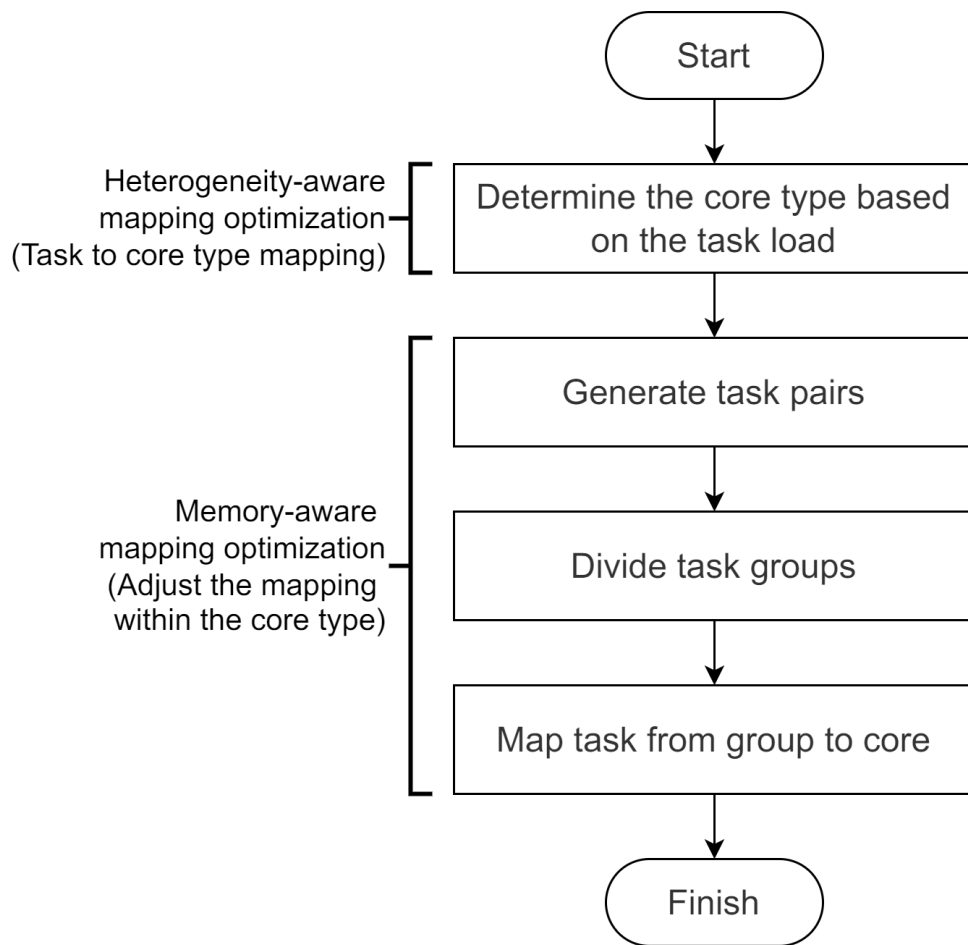


FIGURE 3.5: Heterogeneity-aware priority task mapping process.



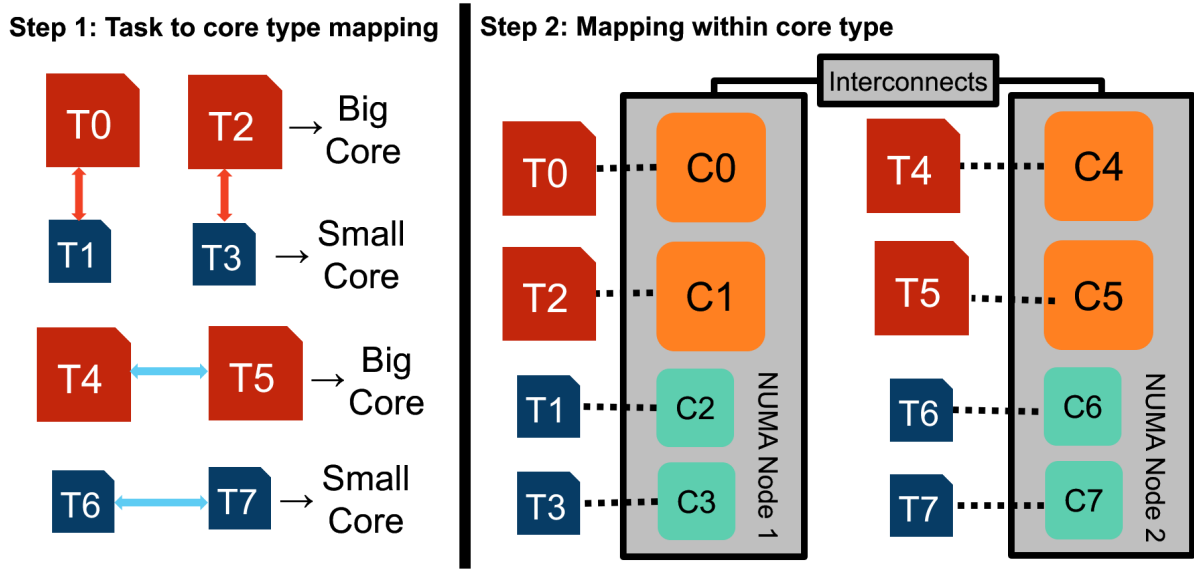


FIGURE 3.6: Heterogeneity-aware priority task mapping example.

performed within the two categories, which will not affect the existing classification results.

The second step is similar to the first and second steps in MPO. Tasks in a category are paired, grouped, and mapped to the cores of the corresponding types on each node. The difference is that, in HPO, the memory-aware mapping is performed within the task category, and thus it will not change the mapping of tasks to core type.

Figure 3.6 shows an example of using HPO to map eight tasks to eight cores. From the task characteristics represented in the figure, it can be seen that the load difference between tasks included in the application is large, and the communication intensity between tasks is small. Therefore the proposed mapping strategy uses HPO for task mapping.

In Step 1 of the example, HPO first decides the core type for each task based on the size of the task load. Tasks with the same core type will be assigned to cores of the corresponding type. For example T0, T2, T4 and T5 in the figure are assigned to Big cores. By prioritizing heterogeneity-aware mapping over memory-aware mapping, HPO can reduce non-optimal mappings like using Small cores for high-load tasks,

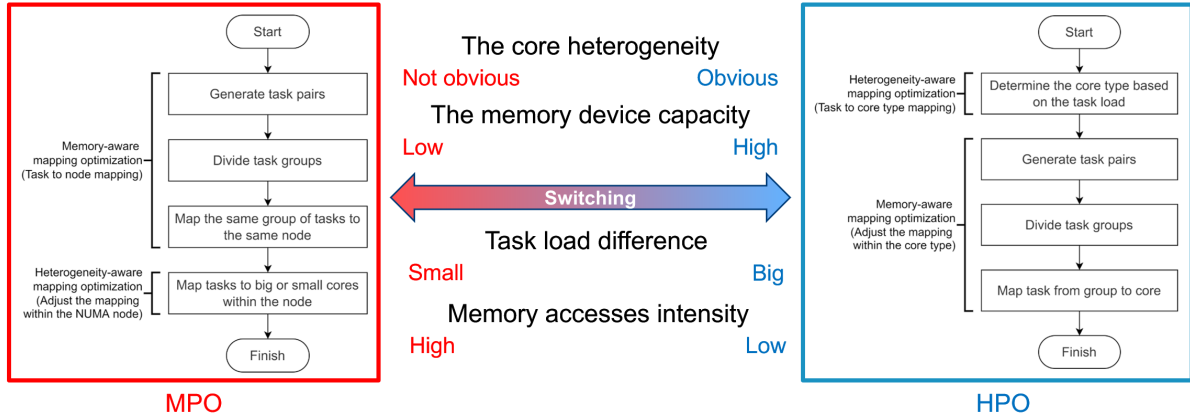


FIGURE 3.7: Priority option switching mechanism concept.

thereby improving overall execution performance.

In Step 2, HPO performs memory-aware mapping by assigning tasks to processor cores within the core type. Without changing the type of cores assigned to tasks, HPO will assign tasks with communication behavior to the same node. For example, if T0, T2, T4 and T5 are assigned to Big core, this step will assign T4 and T5 to the same NUMA node because these two tasks will frequently communicate with each other. However, for T0 and T1, HPO will not change their mapping because they are assigned to different core types. This indeed can cause cross-node communication between tasks, resulting in remote access penalties. However, since the application has a small intensity of communication among tasks, using a non-optimal mapping at this step will not significantly affect the performance.

### 3.4 Priority Option Switching Mechanism

As previously discussed in Sections 3.2 to 3.3, the choice of priority option can significantly affect performance because it determines which factor is more important in the mapping process. Therefore, this thesis proposes a switching mechanism to intelligently select the appropriate mapping priority options for different characteristics of

applications and systems. The design concept of the switching mechanism is shown in Figure 3.7. When the difference between the core in the system and the task load generated by the application is large, the switching mechanism will select HPO first. When the application memory access is intense and the memory device capacity is small, the switching mechanism will select MPO first.

This section first describes some corner cases to illustrate the necessity of switching mapping priorities. First, for memory-aware mapping, when the memory access intensity of the task is much smaller than the amount of memory devices in a NUMA node, the memory-aware task mapping will not have a significant impact on the performance of the application [1]. For the core heterogeneity-aware mapping, even if the hardware in the system is heterogeneous, if the tasks contained in the application are homogeneous, then memory-aware task mapping will not have a significant impact on the performance of the application. These two extreme cases illustrate that using only one mapping mechanism cannot achieve performance gains for all kinds of applications. The evaluation results in Chapter 4 provide evidence justifying this.

As mentioned in Section 3.2, this thesis assumes that some application characteristic information and system configuration information are available. In addition to using this information for the mapping in the above two priority options, this thesis also uses this information to guide the selection of the mapping priority option.

As previously discussed in Section 3.3, the proposed mapping strategy selects one priority option to give precedence to one of the factors to the other. In order to carefully select the appropriate option from the two proposed options, this thesis proposes a priority option switching mechanism (POSM). The mechanism will select the appropriate task mapping priority options based on system configuration and application characteristics. Because this thesis targets NUMA systems with multiple heterogeneous cores in each node, the proposed mechanism uses information about the performance difference between the Big and Small cores in the system and the load characteristics and

communication behaviors of the tasks generated by the application.

In order to quantitatively compare the two priority options, this thesis proposes a metric, called potential benefit (PB), to describe the expected benefit of choosing one of the priority options. The larger the value, the higher the potential benefit of choosing the corresponding priority option.

As shown in Figure 3.8, the workflow of POSM is as follows: After obtaining the information about application characteristics and system configuration, POSM first divides the information into two categories: memory-related and heterogeneity-related. It then uses the information under each classification to calculate memory-aware and heterogeneity-aware PBs separately. The two PB values of M and H correspond to memory-aware PB and heterogeneity-aware PB, respectively. POSM will compare the two values. When M is greater than H, POSM will switch to MPO; when M is less than H, POSM will switch to HPO.

In order to specifically calculate the value of  $M$ , it is necessary to calculate the communication throughput and memory access throughput of the application. The proposed mechanism uses these two values to represent the memory intensiveness of the application. The communication throughput and memory access throughput are defined by Equations (3.1) and (3.2), respectively.  $Scomm_i$  represents the sum of the communication size between task  $i$  and all other tasks.  $Smem_i$  represents the sum of the memory read and write sizes of task  $i$ .  $N$  represents the total number of tasks.  $T_{profile}$  represents the execution time of the application when obtaining the task communication and memory access information.

$$Throughput_{comm} = \frac{\sum_{i=1}^N Scomm_i}{T_{profile}}. \quad (3.1)$$

$$Throughput_{mem} = \frac{\sum_{i=1}^N Smem_i}{T_{profile}}. \quad (3.2)$$

### 3.4. Priority Option Switching Mechanism

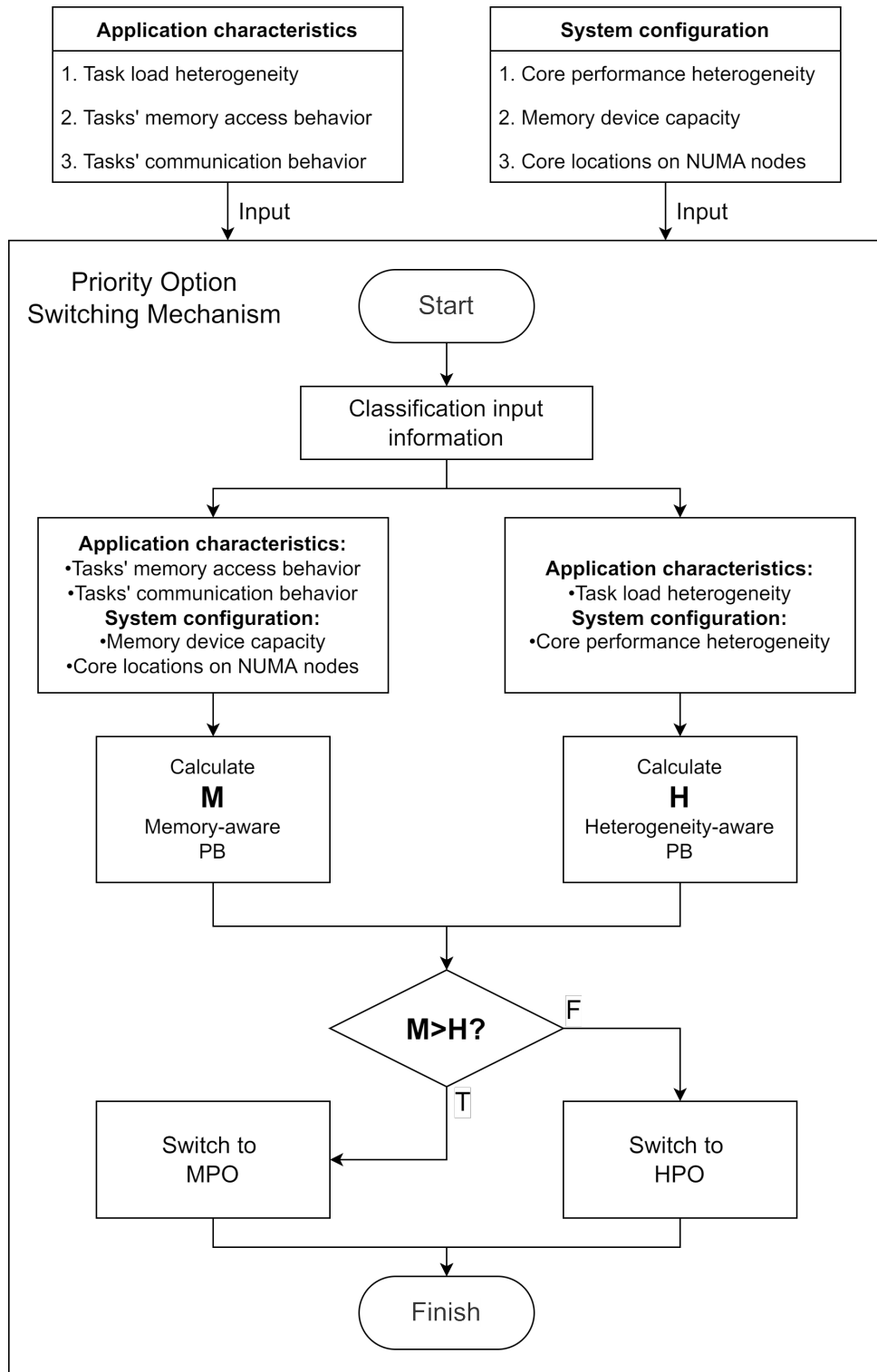


FIGURE 3.8: Priority option switching mechanism process.

Then, the memory-aware PB value,  $M$ , is calculated by

$$M = C_{mem} \times (Throughput_{comm} + Throughput_{mem}), \quad (3.3)$$

where  $C_{mem}$  represents the memory coefficient, which is used to adjust the magnitude of  $M$  so that it can be fairly compared with  $H$ , and to reflect the impact of the memory device capacity in the system on  $M$ .

To calculate the value of  $H$ , it is necessary to calculate the coefficient of variation of the task load of the application. The proposed mechanism uses this value to reflect the load heterogeneity of the tasks in the application. The coefficient of variation of the task load is defined by Equation (3.4).

$$CV_{load} = \frac{\sqrt{\frac{1}{N} \sum_{i=1}^N (L_i - \bar{L})^2}}{\bar{L}}, \quad (3.4)$$

where  $L_i$  represents the load of task  $i$ ,  $\bar{L}$  represents the arithmetic mean of the load of all tasks, and  $N$  represents the total number of tasks. Then, the heterogeneity-aware PB value,  $H$ , is calculated by

$$H = C_{hetero} \times CV_{load}, \quad (3.5)$$

where  $C_{hetero}$  represents the heterogeneity coefficient, which is used to adjust the magnitude of  $H$  so that it can be fairly compared with  $M$ , and is also used to reflect the influence of the core heterogeneity in the system on  $H$ .

## 3.5 Conclusion

This chapter described the proposed mapping strategy in detail, which consists of a mapping strategy with two priority options and a priority option switching mechanism. This chapter explains the workflow of the mapping strategy and switching mechanism and gives mapping examples that correspond to each priority option.

The two priority options processes the task mapping in a specific order so that the secondary factors are optimized without affecting the optimization results of the primary factors. The proposed priority option switching mechanism selects appropriate priority options for individual systems and applications considering their configuration and performance characteristics.

## Chapter 4

# Evaluation

### 4.1 Introduction

This chapter discusses the evaluation environment and results of three evaluation experiments. Section 4.2 introduces the simulation tools and benchmark programs used in the simulation experiments, as well as the simulated system configuration. Section 4.3 conducts preliminary evaluation to provide motivating examples for the proposed method. Section 4.4 quantifies the effects of memory-aware and heterogeneity-aware task mapping strategies using various metrics. Section 4.5 evaluates the performance gain obtained by the proposed POSM as well as its accuracy.

### 4.2 Experimental Environment

This work uses the Sniper [7] simulator to simulate the assumed target system. This simulator is selected because it supports heterogeneous multi-core configurations and allows analyzing the energy efficiency of the system using the McPAT [19] power estimation engine.

By modifying the configuration file to give different parameters of each core in the form of a queue, and setting a certain number of cores to share a memory controller and



L3 cache resources. It is possible to describe a multi-node NUMA system containing heterogeneous multi-core.

As shown in Listing 4.1, the heterogeneous multi-core NUMA system used in the simulation experiments of this thesis contains 16 cores, of which cores 0–3 and 8–11 are Big cores, and their clock frequency is 2.4Ghz; cores 4–7 and 12–15 are Small cores clocked at 1.2GHz. Each core has exclusive L1 and L2 caches. Every eight cores share L3 cache and a memory controller, and can be considered as belonging to the same NUMA node. Therefore, the experimental target system contains two NUMA nodes. Table 4.1 shows the specific processor, cache and memory controller parameters for this system.

LISTING 4.1: Sniper simulator system configuration file.

```
1 ...
2 [perf_model/core]
3 frequency = 2.4
4 frequency [] = , , , 1.2, 1.2, 1.2, 1.2, , , , 1.2, 1.2, 1.2, 1.2
5 ...
6 [perf_model/l3_cache]
7 perfect = false
8 cache_block_size = 64
9 cache_size = 8192
10 associativity = 16
11 address_hash = mask
12 replacement_policy = lru
13 data_access_time = 30
14 tags_access_time = 10
15 perf_model_type = parallel
16 writethrough = 0
17 shared_cores = 8
```

```

18 ...
19 [perf_model/dram]
20 num_controllers = -1
21 controllers_interleaving = 8
22 latency = 45
23 per_controller_bandwidth = 35.76
24 chips_per_dimm = 4
25 dimms_per_controller = 4
26 ...

```

To compare the performance differences between homogeneous systems and heterogeneous systems, this thesis also uses a homogeneous system in simulation experiments. The memory device of this system is similar to the heterogeneous system described earlier, the difference is that this homogeneous system contains 16 identical processor cores, all clocked at 2.4GHz. It contains two NUMA nodes with eight homogeneous cores. Table 4.2 shows the specific processor, cache and memory controller parameters for this system.

The test applications used are the OpenMP version of the fluidanimat kernel from the PARSEC benchmark [6], the lavaMD kernel from the Rodinia benchmark [8], the fft kernel from the splash2 benchmark [24] and the BT and MG kernels from the NAS Parallel Benchmarks [5]. The characteristics of each application are shown in Table 4.3. In terms of application characteristics, the PARSEC-fluidanimat and Rodinia-lavaMD kernels generates threads with different loads, while the splash2-fft kernel generates threads with the same load. The NPB-BT and NPB-MG kernels have obvious memory intensity and can be used for memory-aware experiments.

TABLE 4.1: Heterogeneous simulation configuration.

| Parameter               | Value                                                                                                                                                             |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NUMA nodes (processors) | 1 $\times$ 8-core heterogeneous processor;<br>L1I/L1D cache per core;<br>L2 cache per core;<br>L3 cache shared between 8 cores;<br>1 $\times$ memory controllers; |
| Processor cores         | 8 $\times$ 2.4 GHz cores; 8 $\times$ 1.2 GHz cores;<br>Nehalem performance model                                                                                  |
| L1I/L1D caches          | 256 KB; 8-way; 64-byte line size; LRU policy                                                                                                                      |
| L2 caches               | 2 MB; 8-way; LRU policy                                                                                                                                           |
| L3 caches               | 8 MB; 16-way; LRU policy                                                                                                                                          |
| Memory controllers      | 45 ns latency; 36 GB/s bandwidth;<br>8-way interleaved;<br>DRAM directory model                                                                                   |
| Interconnects           | 25.6 GB/s bandwidth;<br>network bus model                                                                                                                         |

TABLE 4.2: Homogeneous simulation configuration.

| Parameter               | Value                                                                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NUMA nodes (processors) | 1 $\times$ 8-core homogeneous processor;<br>L1I/L1D cache per core;<br>L2 cache per core;<br>L3 cache shared between 8 cores;<br>1 $\times$ memory controller; |
| Processor cores         | 16 $\times$ 2.4 GHz cores;<br>Nehalem performance model                                                                                                        |
| L1I/L1D caches          | 256 KB; 8-way; 64-byte line size; LRU policy                                                                                                                   |
| L2 caches               | 2 MB; 8-way; LRU policy                                                                                                                                        |
| L3 caches               | 8 MB; 16-way; LRU policy                                                                                                                                       |
| Memory controllers      | 45 ns latency; 36 GB/s bandwidth;<br>8-way interleave;<br>DRAM directory model                                                                                 |
| Interconnects           | 25.6 GB/s bandwidth;<br>network bus model                                                                                                                      |

TABLE 4.3: Benchmark application characteristics.

| Benchmark Name      | Task Load Difference | Memory Access Intensity |
|---------------------|----------------------|-------------------------|
| PARSEC-fluidanimate | High                 | Low                     |
| Rodinia-lavaMD      | High                 | Low                     |
| NPB-BT              | High                 | High                    |
| NPB-MG              | Low                  | High                    |
| splash2-fft         | Low                  | Low                     |

### 4.3 Preliminary Evaluation

The goal of this section is to provide the evidence necessary to support the research motivation of this work described in Chapter 1 and the mapping strategy and switching mechanism proposed in Chapter 3. Thus, the preliminary evaluation of this work aims to demonstrate the following three points:

#### 4.3.1 The necessity for heterogeneity

Although the NUMA architecture is very common in modern large HPC systems, heterogeneous multi-core processors have not been widely used in large-scale HPC systems. As mentioned in Section 1.1, multiple processor manufacturers have recently reported the advantages of heterogeneous multi-core processors in terms of energy efficiency. This evaluation demonstrates the performance improvement that can be achieved by introducing heterogeneity by comparing homogeneous and heterogeneous systems with the same configuration except for core frequency. By running the Rodinia-lavaMD kernel with the same mapping on a homogeneous configuration and a heterogeneous configuration, the execution performance of the Rodinia-lavaMD kernel on the two different configurations is compared. Energy efficiency in this work is defined by the following equation:

$$EnergyEfficiency = \frac{IPC}{EnergyConsumption}. \quad (4.1)$$

### 4.3. Preliminary Evaluation

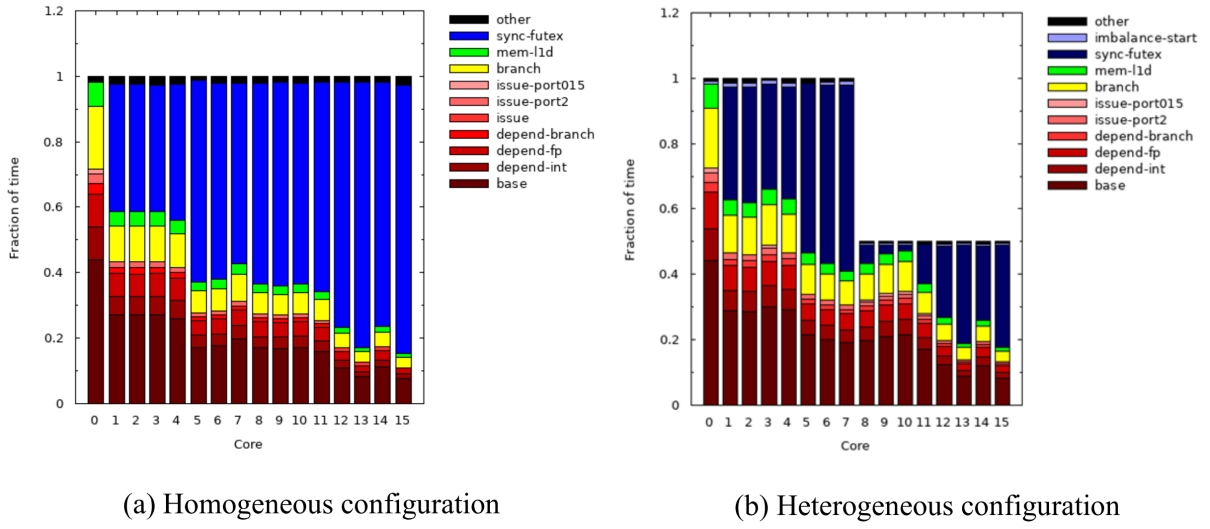


FIGURE 4.1: Comparison of CPI stacks when executing the Rodinia-lavaMD benchmark on homogeneous and heterogeneous systems.

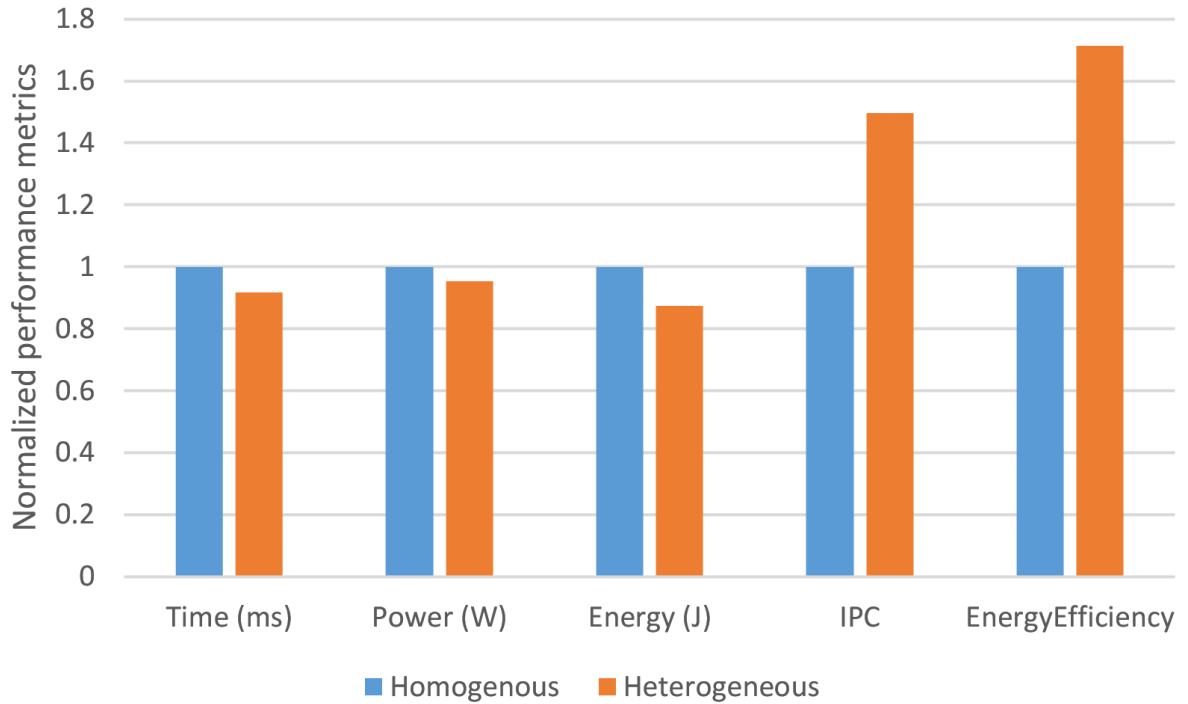


FIGURE 4.2: Performance and energy comparison of homogeneous and heterogeneous systems.

Figure 4.1 compares the CPI stack of the two execution results. In the heterogeneous system, low-load tasks are assigned to the Small cores, thereby greatly reducing the waste of clock cycles due to task load differences. Therefore, in Figure 4.2, which shows the normalized performance metric, it is clear that the heterogeneous configuration increases Instructions Per Cycle (IPC) by reducing the wasted clock cycles, thereby improving energy efficiency.

The results of this experiment illustrate that performance and energy efficiency improvements can be brought about by the introduction of heterogeneous multi-core processor.

### **4.3.2 The importance of suitable task mapping**

This evaluation compares the impact of different mapping strategies on the execution performance of the application. Here, Round-robin, Random, and the proposed heterogeneity-aware mapping strategies are compared in terms of execution time, IPC and energy efficiency using the Rodinia-lavaMD benchmark on the heterogeneous system.

The performance metrics of the random mapping are average of 10 runs. From the normalized performance metric shown in Figure 4.3, it can be seen that, compared with the fixed round-robin, random mapping occasionally produces a better mapping and thus its performance metrics show a small improvement over round-robin. HPO in the proposed mapping strategy can provide suitable task mapping for Rodinia-lavaMD and achieve higher performance and energy efficiency than both round-robin and random mapping.

The results of this experiment illustrate the importance of choosing a suitable mapping for this application.

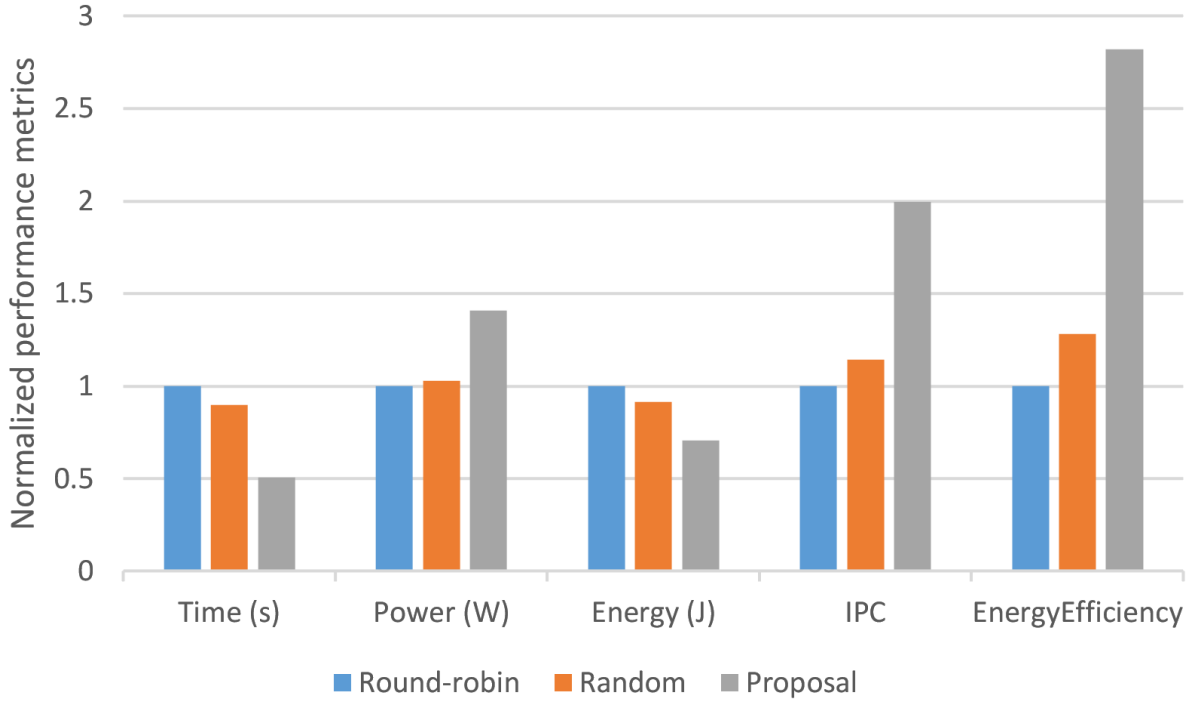


FIGURE 4.3: Performance and energy impacts of using different mapping strategies.

#### 4.3.3 The necessity of priority switching mechanism

The main purpose of this evaluation is to demonstrate the necessity of the two mapping priority options and switching mechanisms proposed in this work. In this experiment, two applications with different characteristics (Rodinia-lavaMD and Splash2-fft) are executed on this system using the proposed HPO. Figure 4.4 shows some evaluation results on a simulator. Compared with the round-robin mapping policy as a baseline, HPO achieves significant improvements in performance and energy efficiency on the Rodinia-lavaMD, while the same is not true on the Splash2-fft.

The results show that the HPO is suitable only for the former application. Similarly, Agung et al. [1] suggest that MPO is suitable only for some type of the applications, but not for all. These results have proven that a single priority option cannot provide a suitable mapping for all kinds of applications. This shows the necessity of the two mapping priority options and the switching mechanism proposed in this work.

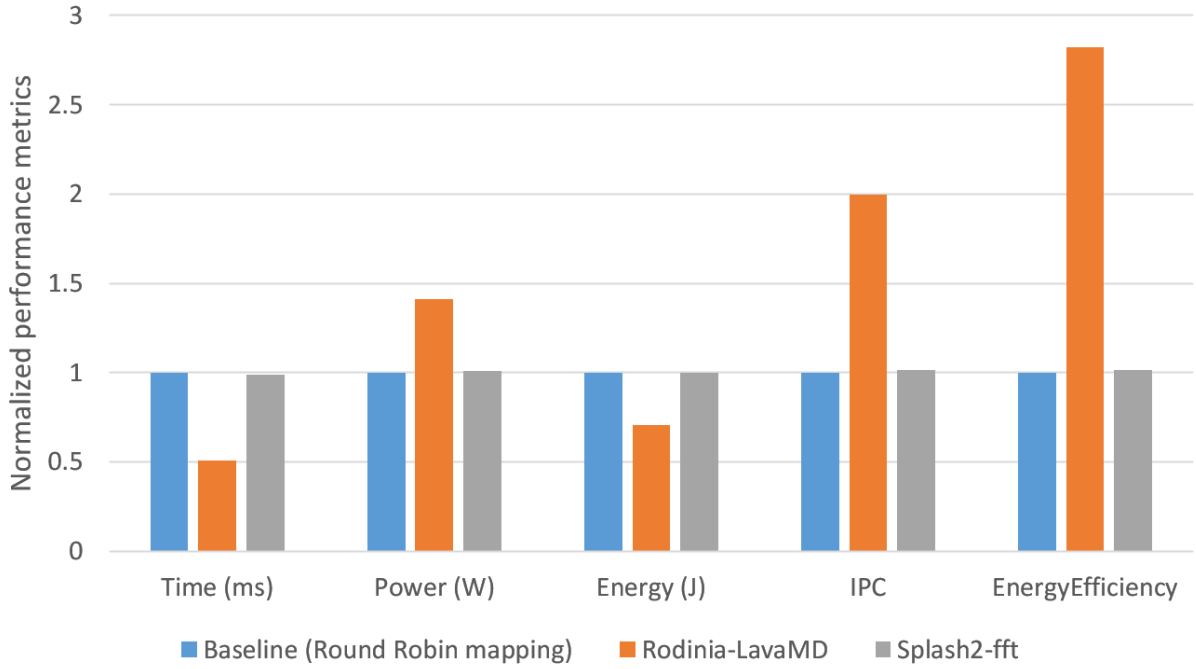


FIGURE 4.4: Performance and energy impacts of using the heterogeneity-aware priority on different applications.

## 4.4 Mapping Strategy Evaluation

The goal of this section is to demonstrate the effectiveness of the mapping strategy proposed in Chapter 3. The two priority options of this mapping strategy include MPO and HPO. This section discusses the effectiveness of these two optimizations separately. To observe the performance improvement resulting from the optimization, this evaluation compares the mapping given by the proposed mapping optimization method to the average of multiple random mappings. The mapping results given by the proposed mapping optimization method or random mapping are stored in a mapping file for the simulator to use.

The overall effectiveness of the mapping strategy is illustrated by comparing a set of performance metrics. The performance metrics observed during the evaluation in this section can be divided into two categories: independent metrics and common metrics. The independent metric is a performance metric directly affected by memory or



#### 4.4. Mapping Strategy Evaluation

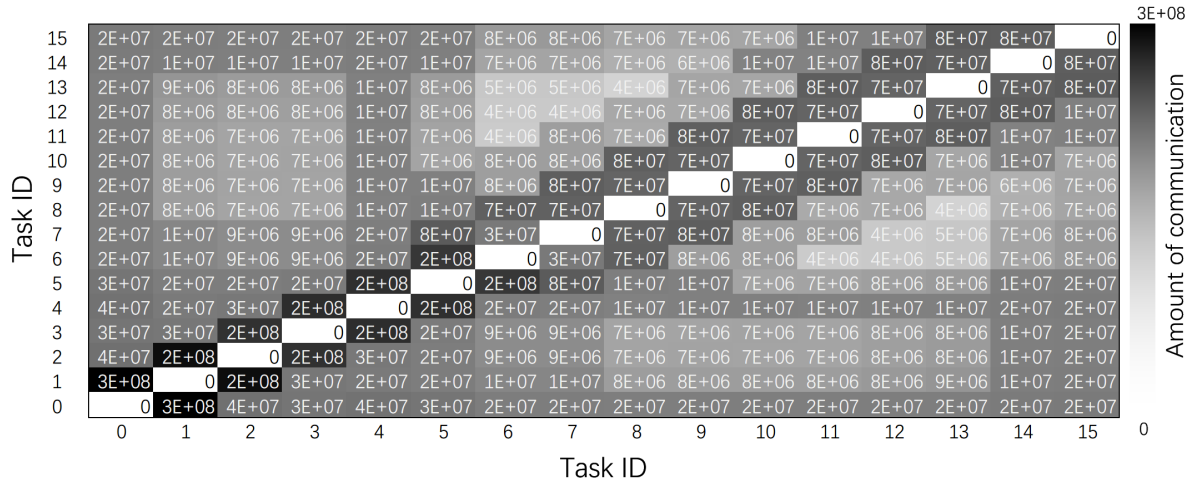


FIGURE 4.5: NPB-BT task communication matrix.

heterogeneity optimization, and can intuitively reflect the performance improvement brought about by the mapping optimization. The common metric is the total execution time of the application, which reflects the overall execution performance of the target application on the target system.

##### 4.4.1 Memory-aware mapping

As mentioned in Section 3.2, performing a memory-aware mapping requires information about the communication behavior between tasks and information about the memory access behavior of the tasks. The total amounts of communication between every pair of tasks and a task communication matrix are shown in Figure 4.5. In addition, the number of reads and writes as well as the total size of reads and writes are shown in Figure 4.6. Using this information and the target system memory device information known from Table 4.1, this mapping optimization can give a mapping result optimized for memory factors. Using an optimized mapping file and a randomly generated random mapping file, the total amount of remote access is estimated. In addition, the execution time of the application can be obtained.

As shown in Figure 4.7, both of the benchmarks, Rodinia-lavaMD and NPB-BT,

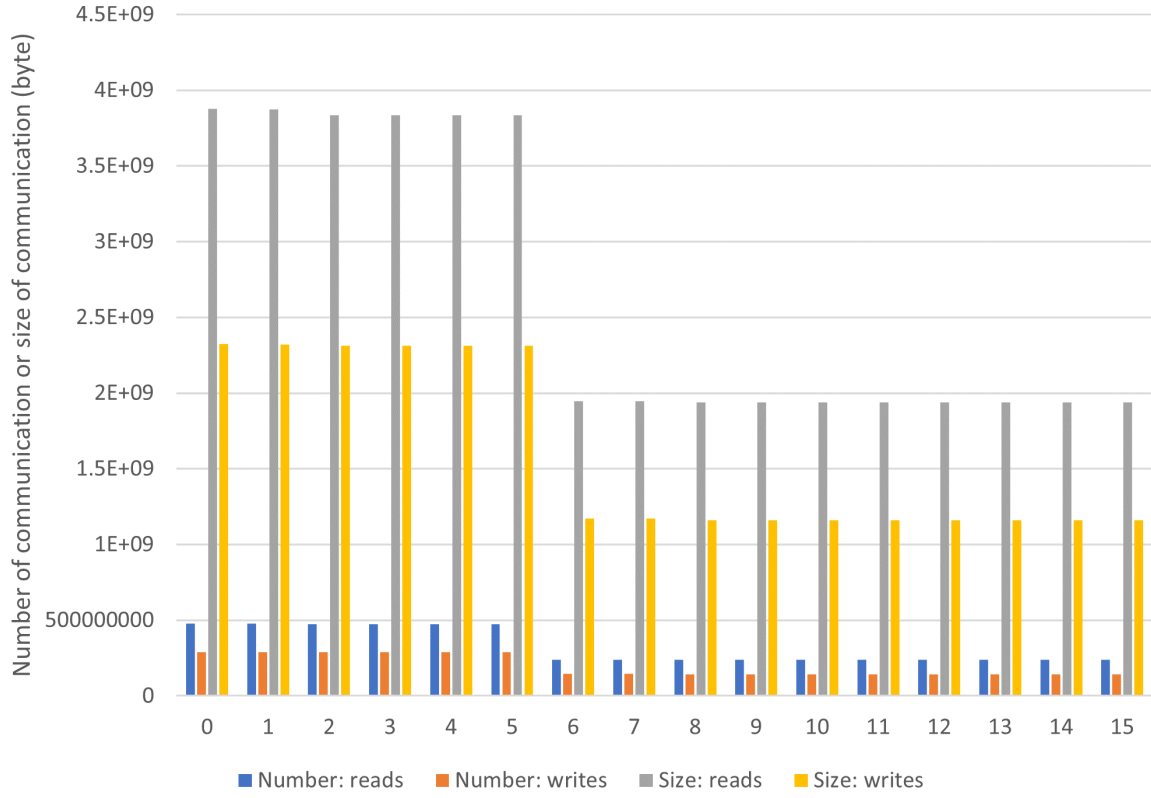


FIGURE 4.6: NPB-BT task memory access behavior.

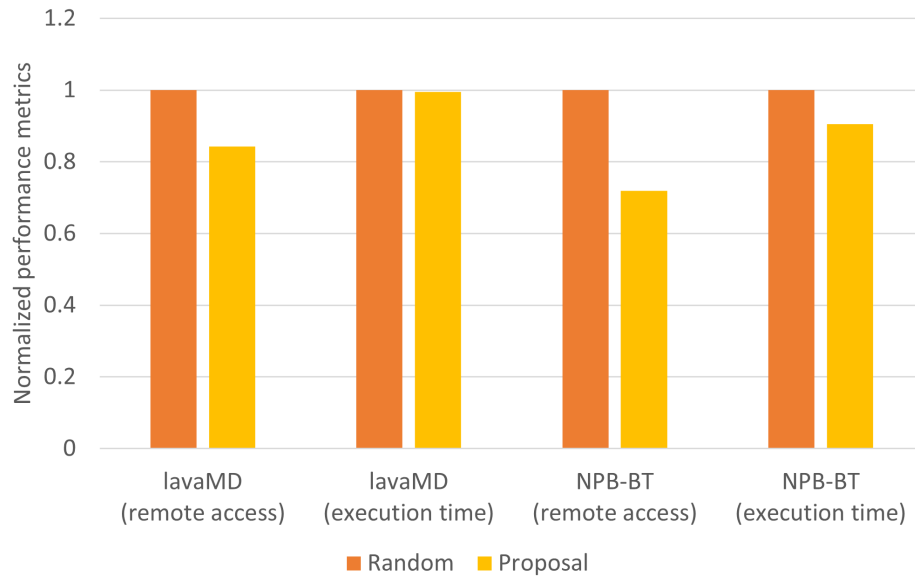


FIGURE 4.7: Performance improvements from memory-aware mapping optimization.

#### 4.4. Mapping Strategy Evaluation

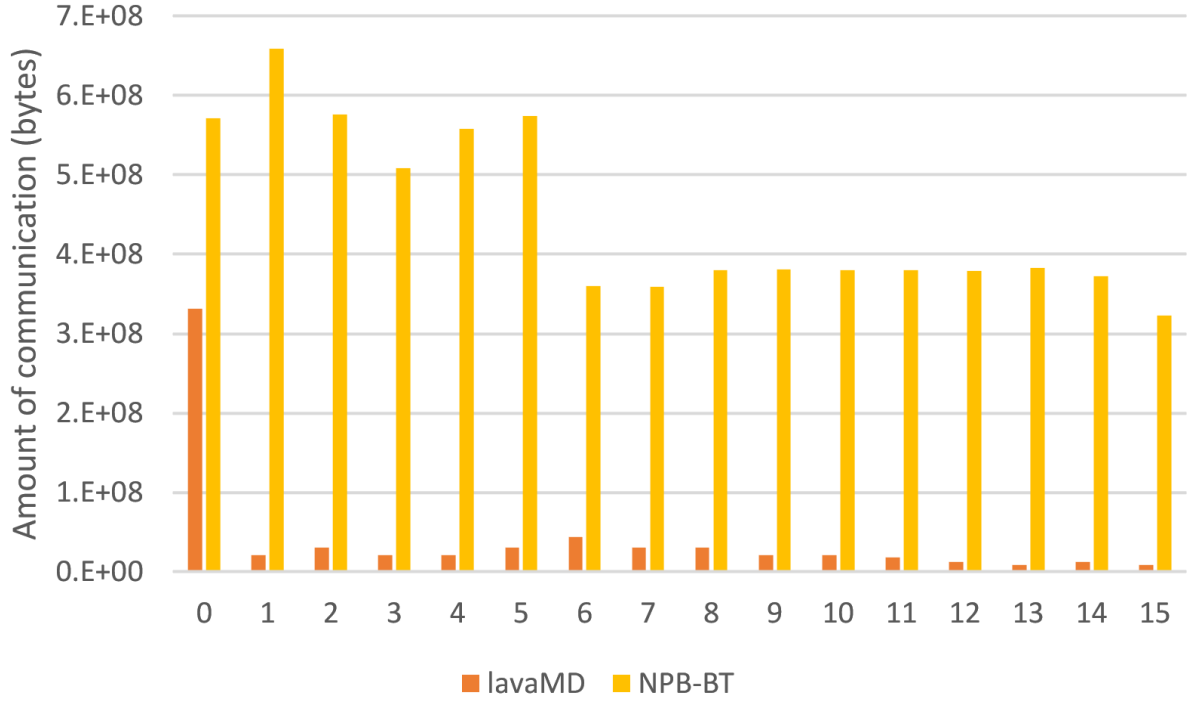


FIGURE 4.8: Comparison of the total communication volume of each task.

can achieve a certain performance improvement on the total amount of remote access. However, Rodinia-lavaMD exhibits almost no change in the execution time, while NPB-BT shows about 10% performance improvement. This is due to the different intensity of memory accesses between the two applications.

The total amounts of communication and memory access performed by the two applications are shown in the Figures 4.8 and 4.9, respectively. It can be seen that the amounts of communication and memory access of Rodinia-lavaMD are much smaller than those of NPB-BT. The communication and memory access intensity of Rodinia-lavaMD does not exceed the capacity of the memory device on a single NUMA node of the target system. Therefore, although Rodinia-lavaMD can achieve a certain reduction of communication, the overall execution time does not significantly reduce.

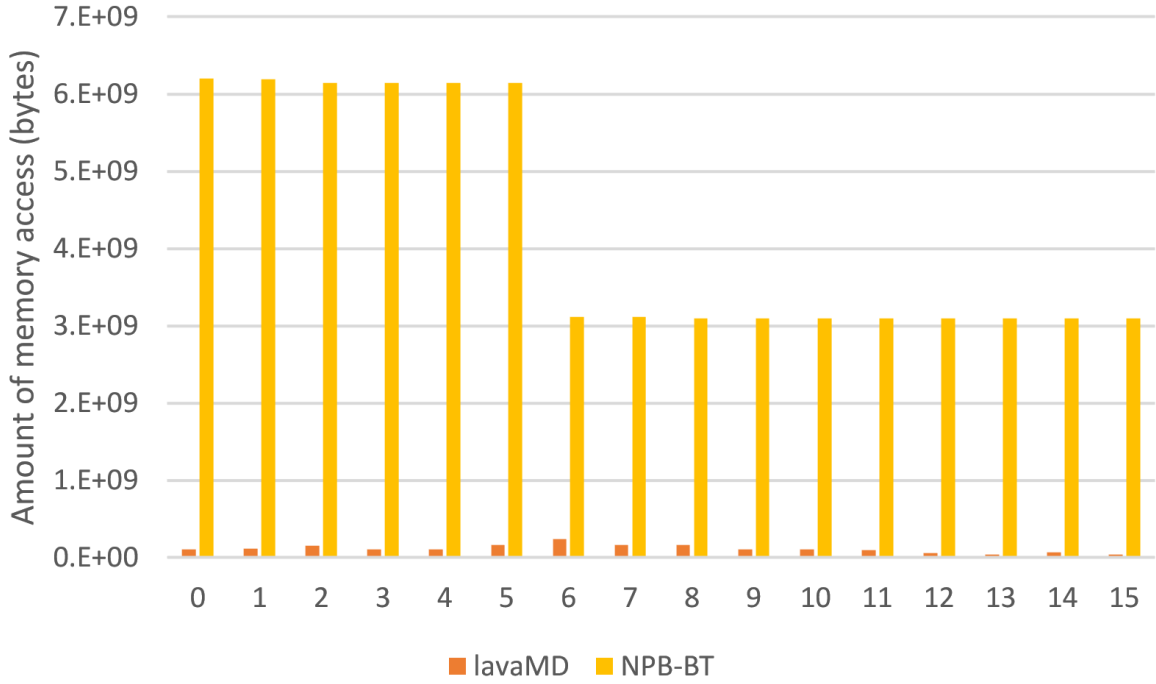


FIGURE 4.9: Comparison of the total amount of memory accessed by each task.

|                          | Core 0    | Core 1    | Core 2    | Core 3    | Core 4   | Core 5    |
|--------------------------|-----------|-----------|-----------|-----------|----------|-----------|
| 1 Instructions           | 80753171  | 88829302  | 111938009 | 81056215  | 75440768 | 119479777 |
| 2 Cycles                 | 194266080 | 194266080 | 194266080 | 194266080 | 97133040 | 97133040  |
| 3 IPC                    | 0.42      | 0.46      | 0.58      | 0.42      | 0.78     | 1.23      |
| 4 Time (ns)              | 80944200  | 80944200  | 80944200  | 80944200  | 80944200 | 80944200  |
| 5 Idle time (ns)         | 62255053  | 60417642  | 55294905  | 62358773  | 46565969 | 25985699  |
| 6 Idle time (%)          | 76.9%     | 74.6%     | 68.3%     | 77.0%     | 57.5%    | 32.1%     |
| 7 Branch predictor stats |           |           |           |           |          |           |
| 8 num correct            | 8708497   | 9848692   | 11717148  | 8791284   | 8014456  | 12716340  |
| 9 num incorrect          | 457251    | 528605    | 598715    | 453559    | 400195   | 672043    |
| 10 misprediction rate    | 4.99%     | 5.09%     | 4.86%     | 4.91%     | 4.76%    | 5.02%     |
| 11 mpki                  | 5.66      | 5.95      | 5.35      | 5.60      | 5.30     | 5.62      |
| 12 TLB Summary           |           |           |           |           |          |           |
| 13 I-TLB                 |           |           |           |           |          |           |
| 14 num accesses          | 8295272   | 9077260   | 11523647  | 8264620   | 7692239  | 12317392  |
| 15 num misses            | 137       | 26        | 26        | 26        | 20       | 26        |
| 16 miss rate             | 0.00%     | 0.00%     | 0.00%     | 0.00%     | 0.00%    | 0.00%     |
| 17 mpki                  | 0.00      | 0.00      | 0.00      | 0.00      | 0.00     | 0.00      |
| 18 D-TLB                 |           |           |           |           |          |           |
| 19 num accesses          | 27680187  | 29956020  | 38551993  | 27560134  | 25846795 | 40901202  |
| 20 num misses            | 573       | 72        | 72        | 65        | 61       | 81        |
| 21 miss rate             | 0.00%     | 0.00%     | 0.00%     | 0.00%     | 0.00%    | 0.00%     |
| 22 mpki                  | 0.01      | 0.00      | 0.00      | 0.00      | 0.00     | 0.00      |
| 23 L2 TLB                |           |           |           |           |          |           |
| 24 num accesses          | 710       | 98        | 98        | 91        | 81       | 107       |
| 25 num misses            | 414       | 97        | 96        | 90        | 80       | 100       |
| 26 miss rate             | 58.31%    | 98.98%    | 97.96%    | 98.90%    | 98.77%   | 93.46%    |
| 27 mpki                  | 0.01      | 0.00      | 0.00      | 0.00      | 0.00     | 0.00      |
| 28 Cache Summary         |           |           |           |           |          |           |
| 29 Cache L1-I            |           |           |           |           |          |           |
| 30 num cache accesses    | 8295272   | 9077260   | 11523647  | 8264620   | 7692239  | 12317392  |
| 31 num cache misses      | 2070      | 200       | 200       | 200       | 159      | 200       |
| 32 miss rate             | 0.02%     | 0.00%     | 0.00%     | 0.00%     | 0.00%    | 0.00%     |
| 33 mpki                  | 0.03      | 0.00      | 0.00      | 0.00      | 0.00     | 0.00      |
| 34                       |           |           |           |           |          |           |

FIGURE 4.10: Sniper profile execution output file.

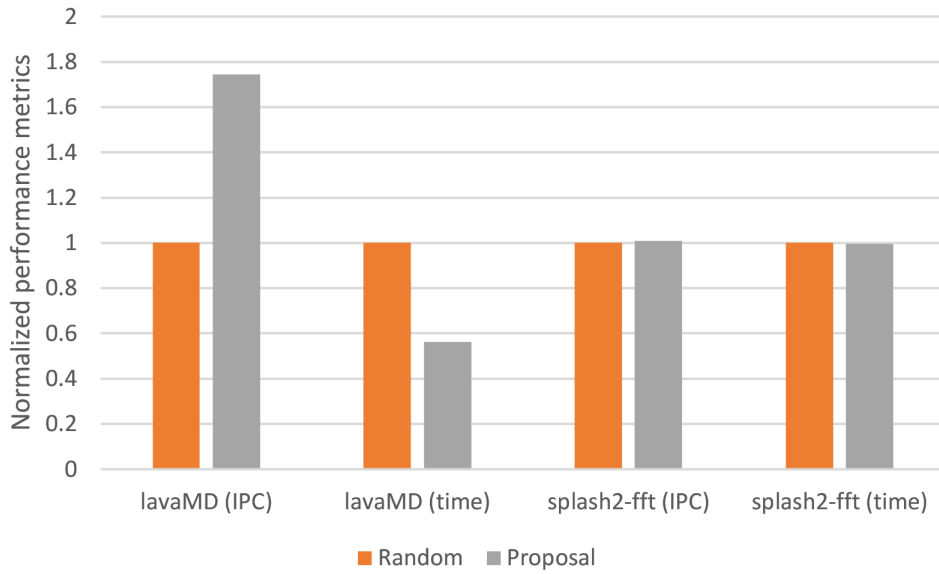


FIGURE 4.11: Performance improvements from heterogeneity-aware mapping optimization.

#### 4.4.2 Heterogeneity-aware mapping

Task load information is required to perform the heterogeneity-aware mapping. As shown in Figure 4.10, after profiling using Sniper, the number of instructions executed on each core can be obtained. Using this information and the target system processor core performance information known from Table 4.1, this map optimization can give a mapping file optimized for heterogeneity. Using an optimized mapping file and randomly generated multiple random mapping files, this evaluation computes relative IPC for the whole execution. Second, using these mapping files as inputs to Sniper, the execution time of the application can be obtained. By comparing these two metrics, we are able to observe the performance improvement obtained by this mapping optimization.

As shown in Figure 4.11, only Rodinia-lavaMD can obtain performance improvements in IPC and execution time. This is due to the very small difference in load between tasks in splash2-fft.

The relative load of each task of the two applications is shown in Figure 4.12. It

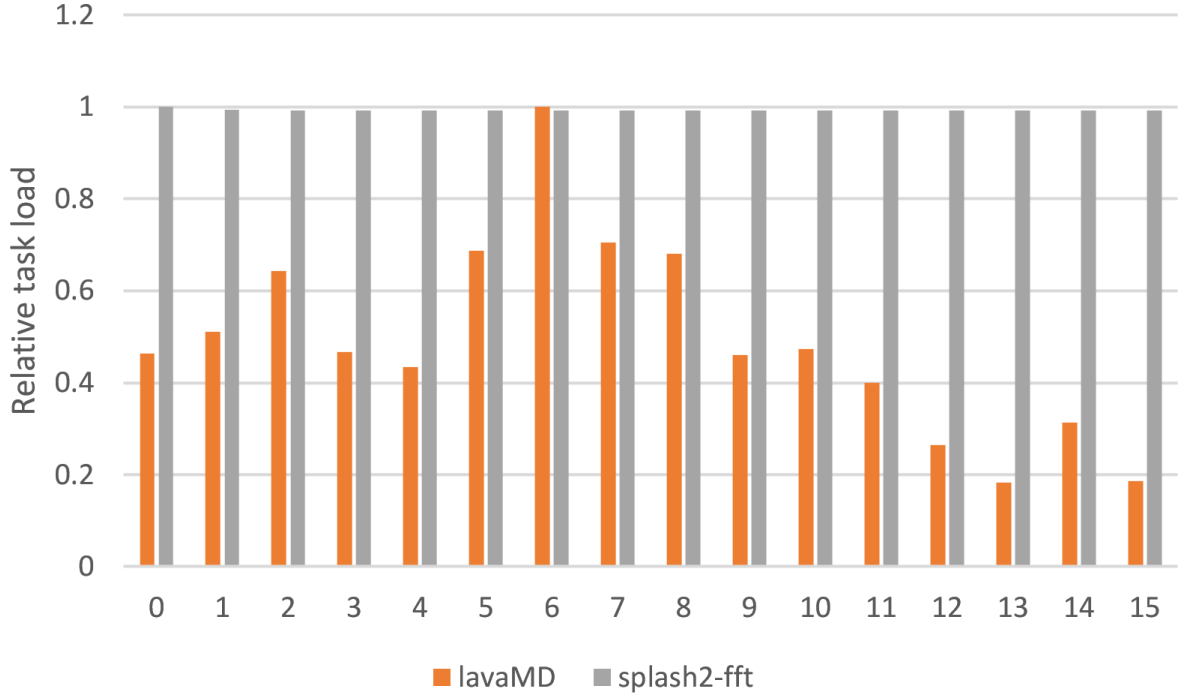


FIGURE 4.12: Comparison of relative tasks load.

can be seen that the load of the tasks included in Rodinia-lavaMD is different, while the load of the tasks included in splash2-fft is consistent. Therefore, even on heterogeneous systems, tuning the task mapping has no impact on the theoretical and practical performance of splash2-fft.

This section described the evaluation principles and results of the two mapping optimizations included in the mapping strategy proposed in Chapter 3. From the results, it can be seen that, for applications with different characteristics, different mapping optimizations are required to obtain performance improvements.

## 4.5 Switching Mechanism Evaluation

The goal of this section is to demonstrate the effectiveness and accuracy of the POSM proposed in Chapter 3. In this evaluation, it is assumed that a heterogeneous multi-core NUMA system sequentially processes a set of applications, one application at a

time. The system chooses the appropriate mapping priority option (MPO or HPO) for each application.

##### 4.5.1 Effectiveness of the Switching mechanism

The following evaluation examines if using the proposed POSM results in an overall performance improvement when dealing with a set of applications. In this evaluation, the application set consists of the following five benchmark kernels: PARSEC-fluidanimate, Rodinia-lavaMD, NPB-BT, NPB-MG, and splash2-fft. First, the simulator uses the default mapping (Packed) to execute the application set and records the execution time of each application. This execution time is used as a baseline to measure the performance improvements measured by other mapping strategies. Then, the application set is executed using a single priority option and the change in execution time is observed.

Figure 4.13 shows the execution time when using the two options normalized by the baseline execution time. It can be seen that three applications, PARSEC-fluidanimate, Rodinia-lavaMD, and NPB-BT, can benefit from HPO, while others can benefit from MPO.

To compare the overall performance of the application set, the sum of the normalized execution times of the five applications is calculated. It can be seen from the sum of normalized execution time shown in Figure 4.14 that, POSM proposed in this thesis can further improve the execution performance compared with task mapping strategies that consider only a single option without priority option switching.

##### 4.5.2 Accuracy of the Switching mechanism

The following evaluation examines if the switching mechanism can select the appropriate priority option for an application after analyzing the application characteristics

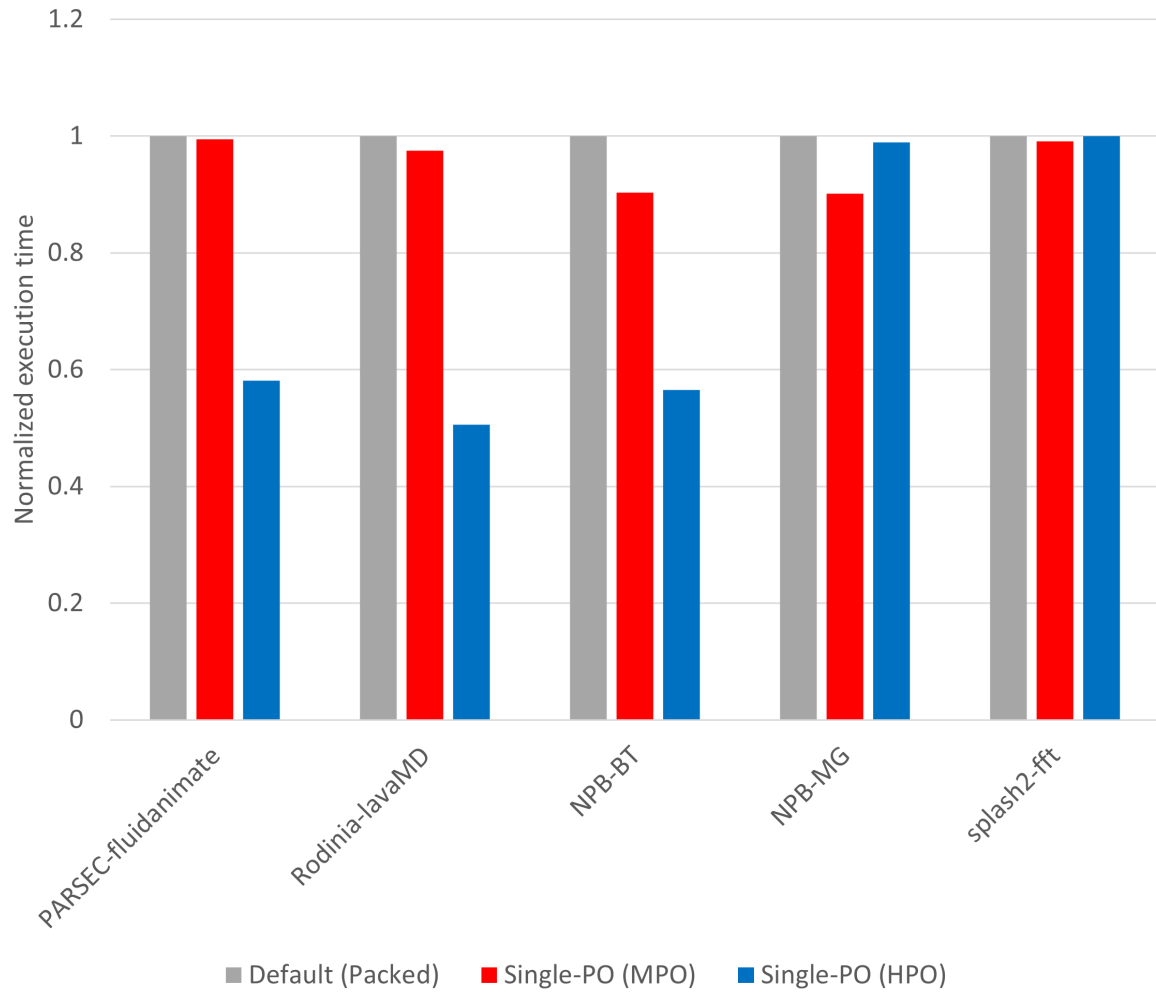


FIGURE 4.13: Normalized execution time for each application.



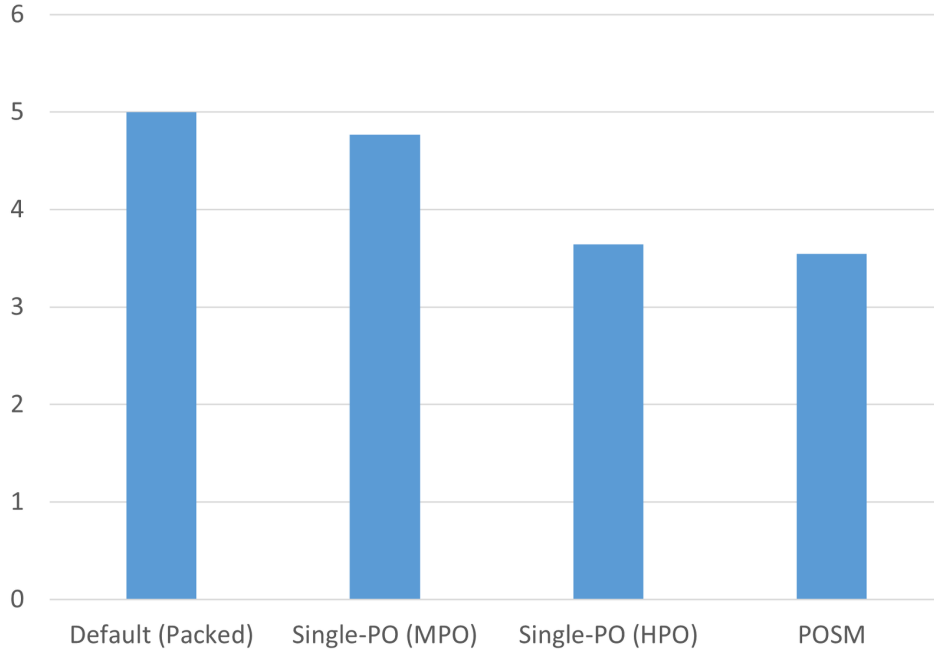


FIGURE 4.14: Sum of normalized execution times.

and system configuration. This evaluation uses the same five benchmark kernels as the previous section as the evaluation application set. From the results in Figure 4.13, the appropriate priority options for each application can be determined. That is, PARSEC-fluidanimate, Rodinia-lavaMD, and NPB-BT should use HPO, while NPB-MG and splash2-fft should use MPO. Since each application needs to be executed twice, this method of determining the optimal priority option is obviously impractical. However, this result can be regarded as an oracle to test the accuracy of POSM proposed in this thesis.

After profiling each application and obtaining the necessary application characteristic information, Equation (3.3) and Equation (3.5) given in Section 3.4 are used to calculate the M and H values for the five applications, respectively.

Figure 4.15 shows M and H normalized by larger one for each application. As mentioned in Section 3.4, POSM proposed in this thesis switches priority options by comparing M and H for each application. POSM selects HPO for PARSEC-fluidanimate,

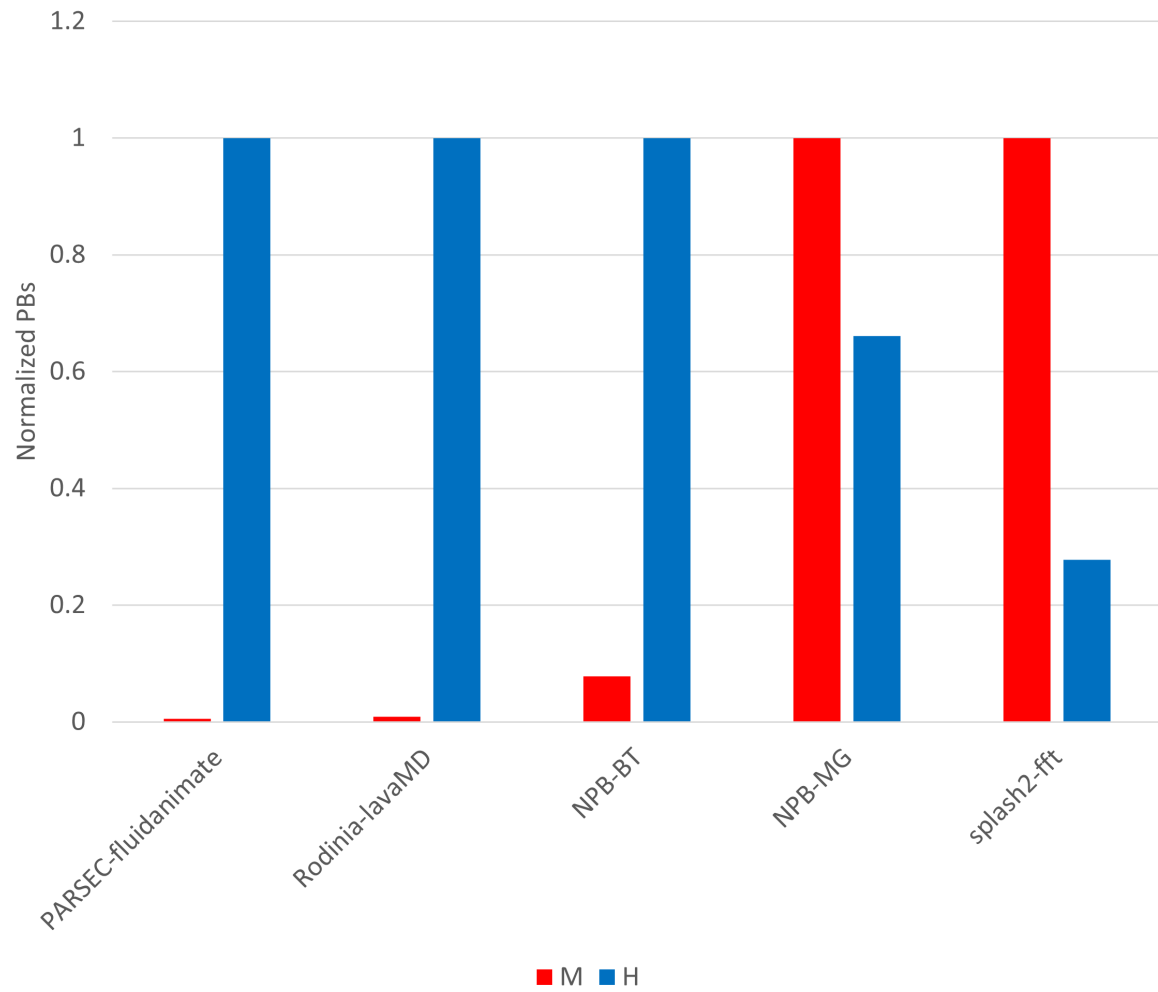


FIGURE 4.15: Normalized PBs for each application.

Rodinia-lavaMD, and NPB-BT, which have large H values. On the other hand, NPB-MG and splash2-fft have large M values, thus MPO is selected. Compared to the above oracle, it can be seen that POSM selects the optimal priority option for each application. Therefore, this evaluation demonstrates that POSM can accurately select the optimal option for the application set.

## 4.6 Conclusion

This chapter presented the evaluation results including the preliminary evaluation to prove the necessity for POSM proposed in this study, the evaluation of the effectiveness of the two priority mapping strategies, and the evaluation of the accuracy of POSM. The evaluation results show that POSM can accurately select appropriate mapping priority options after analyzing the application characteristics and system configurations. After applying the selected mapping priority option for task mapping, the application improves energy efficiency or memory access performance. Compared with other methods that do not switch mapping priorities, the method proposed in this work achieves overall improvements in performance when dealing with a set of applications that exhibits different characteristics.

## Chapter 5

# Conclusions

In the field of HPC, task mapping has long been the focus of research as an important part of computer resource management. The widespread deployment of the NUMA architecture in large-scale HPC systems and its asymmetric access characteristics have spawned many task-mapping studies focusing on memory factors. On the other hand, processors with heterogeneous multi-core architectures have gained a certain market share in personal computing due to their dramatic increase in energy efficiency. Although heterogeneous multi-core processors are not yet widely used in large HPC systems, it is conceivable that future HPC systems will deploy a large number of heterogeneous multi-core processors to simultaneously achieve high peak performance and energy efficiency. Similar to traditional HPC systems, NUMA architecture also needs to be introduced in such systems to provide scalable memory access for a large number of heterogeneous multi-core processors.

Starting from this insight, this thesis discusses new challenges and corresponding solutions to task mapping when these two techniques are combined. In this thesis, two task mapping priority options are proposed: the memory-aware priority option and the heterogeneity-aware priority option. Furthermore, a priority option switching mechanism is proposed. The switching mechanism can select the appropriate priority option for the combination of system and application by analyzing the system configuration and application characteristics. The main contents of this thesis are summarized

as follows.

Chapter 2 introduces two mapping techniques: memory-aware task mapping and heterogeneity-aware task mapping. A review of previous studies related to these two techniques points out that NUMA architectures and heterogeneous multi-core processors are rarely considered in conjunction. Therefore, existing mapping techniques are not able to consider both memory and heterogeneity at the same time in a NUMA system equipped with heterogeneous multi-core processors. As a result, applications are not able to achieve optimal execution performance on such systems. Some of these works are used to provide design ideas for the mapping optimization part of the mapping strategy in this work. This chapter briefly describes the related work of this thesis, and emphasized the differences from the existing studies to clarify the novelty of this thesis. That is, the flexible processing of the mapping using the switching mechanism, and the orderly two-step mapping ensures that the priority of the selected mapping factor.

Chapter 3 describes the mapping technique proposed in this work in detail, including a mapping strategy with two priority options and a priority option switching mechanism. Preliminary experimental results suggest that a single mapping technique cannot cope with a variety of task characteristics and system configurations. Therefore, this work applies two mapping optimizations to task mapping. The two proposed priority options will perform the two mapping optimizations in different orders, and limits the scope of the second optimization step to ensure that it does not interfere with the optimization of high-priority factors already completed in the first step. The interaction of different systems and applications may make the priority of mapping optimization uncertain. To deal with this phenomenon, this work proposes a priority option switching mechanism that analyzes the application characteristics from application profile information and guides the switching mechanism to select the correct

priority option. This chapter presents the detailed workflow and corresponding mapping examples of the mapping strategy and switching mechanism.

Chapter 4 presents the evaluation results. It includes a preliminary evaluation, evaluation of the effectiveness of the two priority mapping strategies, and evaluation of the effectiveness and accuracy of the priority switching mechanism. The preliminary evaluations first demonstrate that the introduction of heterogeneous multi-core processors into the system can yield improvements in energy efficiency. Afterwards, the necessity of the priority switching mechanism is shown by comparing the benefits obtained from applications with different characteristics by single-factor mapping. An evaluation of the effectiveness of the two priority mapping strategies shows the performance improvement obtained by applying the two mapping optimizations separately. Furthermore, a situation where a single mapping optimization cannot obtain significant performance improvement is shown, and the reason behind it is analyzed from the application characteristics. The evaluation of the effectiveness and accuracy of the priority switching mechanism measures the overall performance improvement brought by the switching mechanism when dealing with a set of applications and evaluates whether the analysis for each application in the set is accurate.

There are several future directions for this work. Firstly, subsequent work will not only define different performances for the cores in the system, but also consider disabling and enabling some cores to obtain further performance optimizations. Directly disabling a core may result in less power consumption than using a low-performance core.

Secondly, the mapping technique in this work relies on the information provided by the application profile. To reduce profiling overhead, lightweight and cost-effective methods are needed to analyze application characteristics. In addition, it is necessary to design a more sophisticated heterogeneous mapping considering application characteristics other than task load.

Finally, with the increasing sophistication of modern HPC systems, task mapping may require more consideration. The proposed framework for switching the priority between memory and heterogeneity can be used as a basis to develop task mapping that considers more factors in the future.

# Bibliography

- [1] Mulya Agung, Muhammad Alfian Amrizal, Ryusuke Egawa, and Hiroyuki Takizawa. “DeLoc: A locality and memory-congestion-aware task mapping method for modern numa systems”. In: *IEEE Access* 8 (2020), pp. 6937–6953.
- [2] Mulya Agung, Muhammad Alfian Amrizal, Kazuhiko Komatsu, Ryusuke Egawa, and Hiroyuki Takizawa. “A memory congestion-aware MPI process placement for modern NUMA systems”. In: *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*. 2017, pp. 152–161.
- [3] Hugo Andrade and Ivica Crnkovic. “A review on software architectures for heterogeneous platforms”. In: *2018 25th Asia-Pacific Software Engineering Conference (APSEC)*. 2018, pp. 209–218.
- [4] Apple. *Introducing M1 Pro and M1 Max: the most powerful chips Apple has ever built*. Available (2021). <https://www.apple.com/newsroom/2021/10/introducing-m1-pro-and-m1-max-the-most-powerful-chips-apple-has-ever-built/>. 2021.
- [5] David Bailey, Tim Harris, William Saphir, Rob Van Der Wijngaart, Alex Woo, and Maurice Yarrow. *The NAS parallel benchmarks 2.0*. Tech. rep. NAS-95-020, NASA Ames Research Center, 1995.
- [6] Christian Bienia. “Benchmarking Modern Multiprocessors”. PhD thesis. Princeton University, 2011.
- [7] Trevor E Carlson, Wim Heirman, and Lieven Eeckhout. “Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulation”. In:



- Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. 2011, pp. 1–12.
- [8] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W Sheaffer, Sang-Ha Lee, and Kevin Skadron. “Rodinia: A benchmark suite for heterogeneous computing”. In: *2009 IEEE international symposium on workload characterization (IISWC)*. 2009, pp. 44–54.
- [9] Jian Chen, Nidhi Nayyar, and Lizy K John. “Mapping of applications to heterogeneous multi-cores based on micro-architecture independent characteristics”. In: *Third Workshop on Unique Chips and Systems, ISPASS2007. April 2007*. 2017.
- [10] Mohammad Dashti, Alexandra Fedorova, Justin Funston, Fabien Gaud, Renaud Lachaize, Baptiste Lepers, Vivien Quema, and Mark Roth. “Traffic management: a holistic approach to memory placement on NUMA systems”. In: *ACM SIGPLAN Notices* 48.4 (2013), pp. 381–394.
- [11] Matthias Diener, Eduardo HM Cruz, Marco AZ Alves, Mohammad S Alhakeem, Philippe OA Navaux, and Hans-Ulrich Hei. “Locality and balance for communication-aware thread mapping in multicore systems”. In: *European Conference on Parallel Processing*. Springer. 2015, pp. 196–208.
- [12] Jiun-Hung Ding, Ya-Ting Chang, Zhou-dong Guo, Kuan-Ching Li, and Yeh-Ching Chung. “An efficient and comprehensive scheduler on Asymmetric Multicore Architecture systems”. In: *Journal of Systems Architecture* 60.3 (2014), pp. 305–314.
- [13] Fabien Gaud, Baptiste Lepers, Justin Funston, Mohammad Dashti, Alexandra Fedorova, Vivien Quéma, Renaud Lachaize, and Mark Roth. “Challenges of memory management on modern NUMA systems”. In: *Communications of the ACM* 58.12 (2015), pp. 59–66.

- [14] Intel. *Intel Architecture Day 2021 Presentation*. (2021). <https://download.intel.com/newsroom/2021/client-computing/intel-architecture-day-2021-presentation.pdf>. 2021.
- [15] Emmanuel Jeannot, Guillaume Mercier, and François Tessier. “Process placement in multicore clusters: Algorithmic issues and practical techniques”. In: *IEEE Transactions on Parallel and Distributed Systems* 25.4 (2013), pp. 993–1002.
- [16] David Koufaty, Dheeraj Reddy, and Scott Hahn. “Bias scheduling in heterogeneous multi-core architectures”. In: *Proceedings of the 5th European conference on Computer systems*. 2010, pp. 125–138.
- [17] Rakesh Kumar, Keith I Farkas, Norman P Jouppi, Parthasarathy Ranganathan, and Dean M Tullsen. “Single-ISA heterogeneous multi-core architectures: The potential for processor power reduction”. In: *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36*. IEEE. 2003, pp. 81–92.
- [18] Christoph Lameter. “An overview of non-uniform memory access”. In: *Communications of the ACM* 56.9 (2013), pp. 59–54.
- [19] Sheng Li, Jung Ho Ahn, Richard D Strong, Jay B Brockman, Dean M Tullsen, and Norman P Jouppi. “McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures”. In: *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture*. 2009, pp. 469–480.
- [20] Teng Ma, Thomas Herault, George Bosilca, and Jack J Dongarra. “Process distance-aware adaptive MPI collective communications”. In: *2011 IEEE International Conference on Cluster Computing*. IEEE. 2011, pp. 196–204.
- [21] Chris A Mack. “Fifty years of Moore’s law”. In: *IEEE Transactions on semiconductor manufacturing* 24.2 (2011), pp. 202–207.

- [22] Jason A Poovey, Michael C Rosier, and Thomas M Conte. “Pattern-Aware Dynamic Thread Mapping Mechanisms for Asymmetric Manycore Architectures”. In: 2011.
- [23] Li Wang, Minqi Zhou, Zhenjie Zhang, Ming-Chien Shan, and Aoying Zhou. “Numa-aware scalable and efficient in-memory aggregation on large domains”. In: *IEEE Transactions on Knowledge and Data Engineering* 27.4 (2014), pp. 1071–1084.
- [24] Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. “The SPLASH-2 programs: Characterization and methodological considerations”. In: *ACM SIGARCH computer architecture news* 23.2 (1995), pp. 24–36.
- [25] Jin Zhang, Jidong Zhai, Wenguang Chen, and Weimin Zheng. “Process mapping for MPI collective communications”. In: *European Conference on Parallel Processing*. Springer. 2009, pp. 81–92.

# Publication

[1] Yifan Jin, Mulya Agung, Jiaheng Liu, Hiroyuki Takizawa, "Memory-aware Task Mapping for Heterogeneous Multi-Core Systems," HPC Asia 2022 (Poster presentation), 12 Jan. 2022, Kobe, Japan.

# Acknowledgements

First and foremost, I would like to express my appreciation to my supervisor, Professor Hiroyuki Takizawa. Without his assistance and dedicated involvement in every step throughout the process, this thesis would have never been accomplished. His immense knowledge and plentiful experience have encouraged me in all the time of my academic research and daily life.

I would also like to show gratitude to my committee, including Professor Hiroaki Kobayashi and Associate Professor Masayuki Sato. I discussed early versions of this thesis with them. They gave many thoughtful comments and recommendations in our discussion. The impact of their enthusiastic help is obvious throughout this thesis.

I would also like to express my appreciation to Professor Ryusuke Egawa (Tokyo Denki University), Associate Professor Yoichi Shimomura, Assistant Professor Keichi Takahashi, and Doctor Mulya Agung, their warm encouragement and valuable suggestions have brought great help to my thesis work.

Additionally, I want to express my heartfelt thanks to all the members in Takizawa laboratory, working with them was an extraordinary experience.

Finally, thanks to my parents for creating good economic conditions and a harmonious family atmosphere for me. This thesis stands as a testament to their unconditional love and encouragement.

Yifan JIN

January 27, 2022